WEEK 2 · LECTURE

Counting, dictionaries & measurement validity

Dr. Philipp K. Masur
Computational Analysis of Digital Communication · Vrije Universiteit Amsterdam

Three tweets, one word list

TWEET 1

With visual assistance, this is a good source of information in how mRNA vaccines work #COVID19 #Corona

TWEET 2

Mind-boggling. #Corona is a criminal conspiracy, just follow the patents.

TWEET 3

If the masks work, why the 6 feet? If the 6 feet works, why the masks? If they both work, why lockdown? #COVID19 #Corona

Three tweets, one word list

TWEET 1

With visual assistance, this is a **good** source of information in how mRNA vaccines **work** #COVID19 #Corona

TWEET 2

Mind-boggling. #Corona is a **criminal conspiracy,** just follow the patents.

TWEET 3

If the masks **work,** why the 6 feet? If the 6 feet **works,** why the masks? If they both **work,** why lockdown? #COVID19 #Corona

Three tweets, one word list

	+	-
TWEET 1 With visual assistance, this is a **good** source of information in how mRNA vaccines **work** #COVID19 #Corona	+2	-0
TWEET 2 Mind-boggling. #Corona is a **criminal conspiracy,** just follow the patents.	+0	-2
TWEET 3 If the masks **work,** why the 6 feet? If the 6 feet **works,** why the masks? If they both **work,** why lockdown? #COVID19 #Corona	+3	-0

Three tweets, one word list

	+	-	THE LIST SAYS
TWEET 1 With visual assistance, this is a **good** source of information in how mRNA vaccines **work** #COVID19 #Corona	+2	-0	POSITIVE
TWEET 2 Mind-boggling. #Corona is a **criminal conspiracy,** just follow the patents.	+0	-2	NEGATIVE
TWEET 3 If the masks **work,** why the 6 feet? If the 6 feet **works,** why the masks? If they both **work,** why lockdown? #COVID19 #Corona	+3	-0	POSITIVE

Three tweets, one word list

	+	-	THE LIST SAYS	YOU SAID
TWEET 1 With visual assistance, this is a **good** source of information in how mRNA vaccines **work** #COVID19 #Corona	+2	-0	POSITIVE	POSITIVE
TWEET 2 Mind-boggling. #Corona is a **criminal conspiracy,** just follow the patents.	+0	-2	NEGATIVE	NEGATIVE
TWEET 3 If the masks **work,** why the 6 feet? If the 6 feet **works,** why the masks? If they both **work,** why lockdown? #COVID19 #Corona	+3	-0	POSITIVE	NEGATIVE

Three tweets, one word list

	TWEET	+	−	THE LIST SAYS	YOU SAID
	TWEET 1 With visual assistance, this is a **good** source of information in how mRNA vaccines **work** #COVID19 #Corona	+2	−0	POSITIVE	POSITIVE
	TWEET 2 Mind-boggling. #Corona is a **criminal conspiracy,** just follow the patents.	+0	−2	NEGATIVE	NEGATIVE
	TWEET 3 If the masks **work,** why the 6 feet? If the 6 feet **works,** why the masks? If they both **work,** why lockdown? #COVID19 #Corona	+3	−0	POSITIVE	NEGATIVE

Three words from the list, one hostile tweet. The count is right about the words and wrong about the text — and nothing in the number says so.

Last week you learned to count

- A text became a row of the **document–term matrix**: one column per word, a count in every cell, and importantly: the order of the words gone
- That is a representation, but not yet a classification: counting itself does not provide a label (e.g., *negative* or *migration*)
- A measurement or classification adds two things a count lacks: a **claim about meaning**, and a **check** that the claim holds
- This week supplies the “oldest way” of making such a claim, and the general approach of checking it (validation)

> **Counting → measuring is the step from step 02 of the pipeline to steps 03 and 04.**

The second question is always relevant

1. How can we analyse all of this text — automatically?
2. **How can we make sure that the analysis is valid?**

This lecture

I

From counting to measuring

Reminder: How to count words, and what a count can and cannot say

II

Dictionaries

The first automated classification: a word list and a counting rule

III

Validation

Gold standards, agreement, the confusion matrix, and the numbers you need to report

IV

Dictionaries in research

Two studies, and when a word list is the right tool

V

This week

Labs, the gold set you build on Thursday, and homework 2

I · From counting to measuring

Reminder: How to count words in a corpus

```
1  library(sotu)
2
3  sotu_speeches <- as_tibble(sotu_meta) |>
4    rename(doc_id = X) |>
5    mutate(text = sotu_text)
6
7  sotu_speeches |>
8    select(doc_id, president, year, text)
```

- Every State of the Union address, 1790 to 2020: 240 texts
- One row per speech, metadata beside the text
- The shape you already know from last week's lab B

OUTPUT

```
# A tibble: 240 × 4
  doc_id president          year text
   <int> <chr>             <int> <chr>
1      1 George Washington  1790 "Fellow-Citizens of the Se..."
2      2 George Washington  1790 "\n\n Fellow-Citizens of the..."
3      3 George Washington  1791 "\n\n Fellow-Citizens of the..."
4      4 George Washington  1792 "Fellow-Citizens of the Se..."
5      5 George Washington  1793 "\n\n Fellow-Citizens of the..."
6      6 George Washington  1794 "\n\n Fellow-Citizens of the..."
7      7 George Washington  1795 "\n\nFellow-Citizens of the ..."
# ℹ 233 more rows
```

> A speech is 8,000 words on average. Two hundred and forty of them is two million tokens.

Kahle, D. (2020). *sotu: United States presidential State of the Union addresses* (R package, version 1.0.2). https://CRAN.R-project.org/package=sotu

Reminder: The corpus as a matrix

```
sotu_dtm <- sotu_speeches |>
  unnest_tokens(word, text) |>
  count(doc_id, word) |>
  pivot_wider(names_from = word,
              values_from = n, values_fill = 0)

dim(sotu_dtm)

mean(sotu_dtm[-1] == 0)     # share of empty cells

sotu_dtm |>
  select(doc_id, freedom, war, tax, terror) |>
  filter(doc_id %in% c(1, 74, 153, 222, 235))
```

OUTPUT

```
[1]   240 30586
```

```
[1] 0.9411686
```

```
# A tibble: 5 × 5
  doc_id freedom   war   tax terror
   <int>   <int> <int> <int>  <int>
1      1       0     1     0      0
2     74       3    20     1      0
3    153      10    31     0      0
4    222      10    12     7     13
5    235       1     4     3      1
```

- Last week's matrix, again built the same way: 240 rows, one column per distinct word (plus the id)
- Too wide to print, so we look at four columns (*freedom*, *war*, *tax*, *terror*) for five speeches
- The same word can be frequent in one speech and absent in another: that variation is what the next slides use

> **Ninety-four cells in a hundred are zero. Every count on the next slides is a column sum or a row sum of this matrix.**

The most frequent words are the least interesting

ALL TOKENS

```
# A tibble: 8 × 2
  word      n
  <chr> <int>
1 the   165601
2 of    106402
3 and    68063
4 to     68037
5 in     43429
6 a      31342
7 that   24113
8 for    21701
```

AFTER `anti_join(stop_words)`

```
# A tibble: 8 × 2
  word          n
  <chr>     <int>
1 government  7597
2 congress    5808
3 united      5156
4 people      4298
5 country     3641
6 public      3419
7 time        3188
8 american    2988
```

- **Stop words** carry grammar, not content: *the*, *of*, *and* are a fifth of the corpus
- Removing them is common practice, but *not* is on most lists, and week 1's sandbox showed what that costs

> Every cleaning step is a decision about which words are allowed to count.

Trimming the long tail

```
sotu_tokens |> distinct(word) |> nrow()

sotu_tokens |>
  count(word) |>
  filter(n >= 100) |>
  nrow()
```

OUTPUT

```
types_total types_100plus
      30585          2108
```

- Thirty thousand distinct words, and most occur a handful of times: names, numbers, misspellings
- Two thousand words account for almost all of the two million tokens
- Dropping rare terms makes the matrix smaller and models more stable, but it also loses the rare event we may be looking for

> **This distribution has a name, Zipf's law, and it appears in nearly every language and corpus.**

Stemming or lemmatising

```
sotu_tokens |>
  filter(word %in% c("nations", "governing", "better",
                     "countries", "was")) |>
  distinct(word) |>
  mutate(stem  = SnowballC::wordStem(word),
         lemma = textstem::lemmatize_words(word))
```

- A **stem** is cut by rule: fast, language-agnostic, sometimes not a word
- A **lemma** is looked up: the dictionary form, so *was* becomes *be*
- For a word list (and thus for the dictionary approach) you want lemmas: the list contains *country*, not *countri*

OUTPUT

```
# A tibble: 5 × 3
  word      stem    lemma
  <chr>     <chr>   <chr>
1 was       wa      be
2 nations   nation  nation
3 better    better  good
4 countries countri country
5 governing govern  govern
```

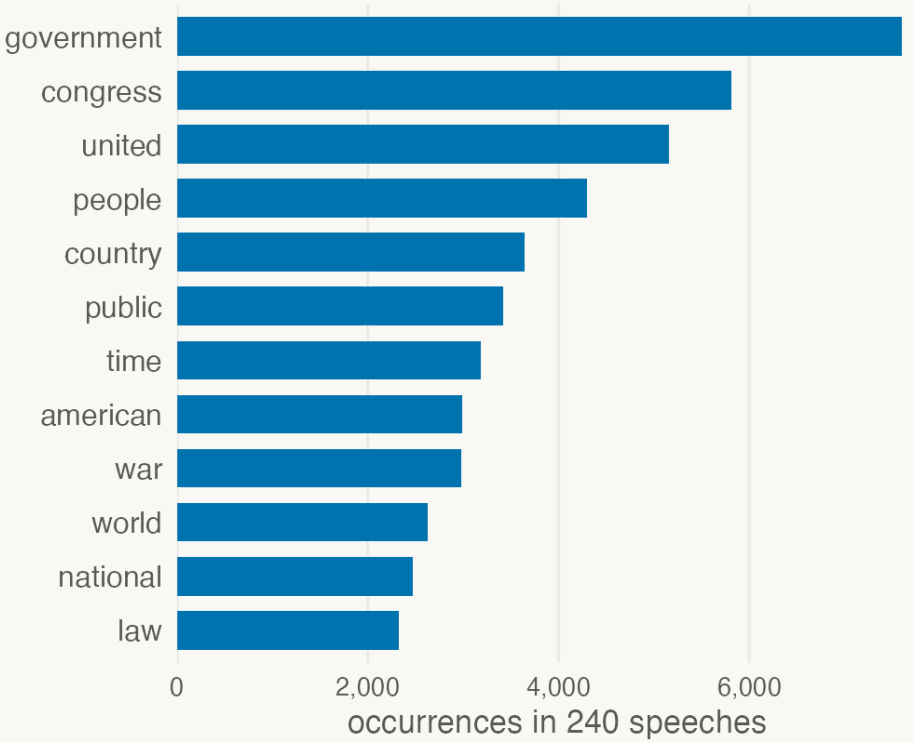
One sentence, two kinds of normalising

SENTENCE	the	states	were	governing	their	countries	better
↓ `SnowballC::wordStem()` cuts the ending by rule							
STEMMED	the	state	were	govern	their	countri	better
↓ `textstem::lemmatize_words()` looks the word up							
LEMMATISED	the	state	be	govern	their	country	good

> The stemmer leaves *were* and *better* alone and clips *countries* to a non-word.
> The lemmatiser returns *be*, *good* and *country*.

The same counts, as a cloud and as bars





> Both show absolute frequency in the whole corpus. The cloud looks good, but only the bars let you compare the words.

How tf-idf weights a word

- Raw counts reward words every speech uses, like *government* and *congress*
- So counts alone cannot show which words set one president apart
- **tf** is the share of a president's words that a word makes up
- **idf** is ln(4 ÷ presidents who use the word): 0 if all four use it, 1.39 if only one does
- **tf-idf** is tf × idf: high only for a word that is frequent here and rare elsewhere

> ***america*** **is Obama's most frequent word and scores zero, because all four presidents say it.**

FOUR WORDS IN OBAMA'S SPEECHES

```
# A tibble: 4 × 6
  word            n       tf presidents   idf   tf_idf
  <chr>       <int>    <dbl>      <int> <dbl>    <dbl>
1 isil            9 0.000438          1 1.39  0.000608
2 industry       24 0.00117           2 0.693 0.000810
3 afghanistan    23 0.00112           3 0.288 0.000322
4 america       204 0.00994           4 0     0
```

What makes a president's speeches distinctive

```
sotu_tfidf <- sotu_tokens |>
  filter(president %in% c("Barack Obama", "Donald Trump",
                          "George W. Bush", "William J. Clinton")) |>
  anti_join(stop_words, by = "word") |>
  count(president, word) |>                    # term frequency, per president
  bind_tf_idf(word, president, n)              # ... weighted by how few presidents use the word
```

- All speeches of one president are pooled into a single document, so the idf compares four presidents
- Raw counts would put *america*, *people* and *congress* on top for everyone
- `bind_tf_idf()` adds tf, idf and tf-idf as columns next to the counts

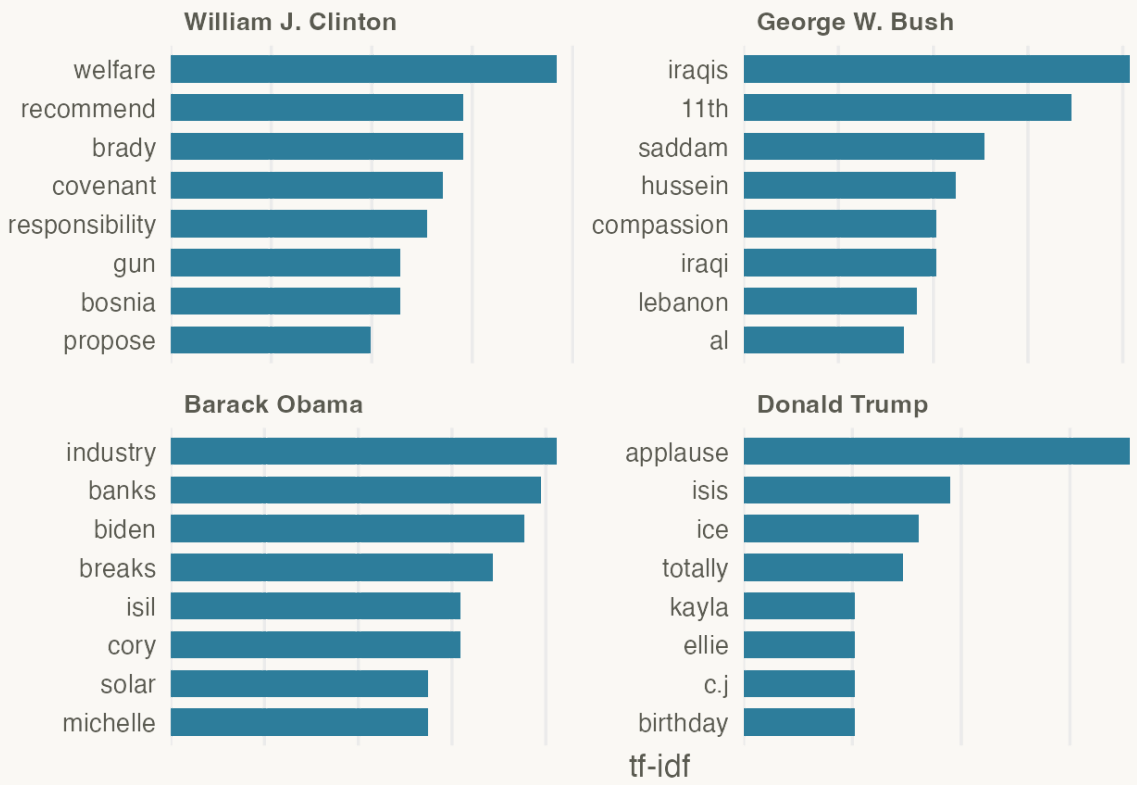
> **The first weighting you meet. It sees only how unevenly a word is spread, not what it means.**

Silge, J., & Robinson, D. (2017). *Text mining with R: A tidy approach*. O'Reilly. Chapter 3.

What makes a president's speeches distinctive

```
sotu_tfidf |>
  group_by(president) |>
  slice_max(tf_idf, n = 8) |>
  ggplot(aes(tf_idf,
      reorder_within(word, tf_idf, president))) +
  geom_col() +
  facet_wrap(~ president, scales = "free_y")
```

- Each panel is one president's most distinctive words
- The crises and the enemies show up before any policy does
- *applause* is not a word Trump said: it is the transcript's stage direction, and tf-idf cannot tell the difference

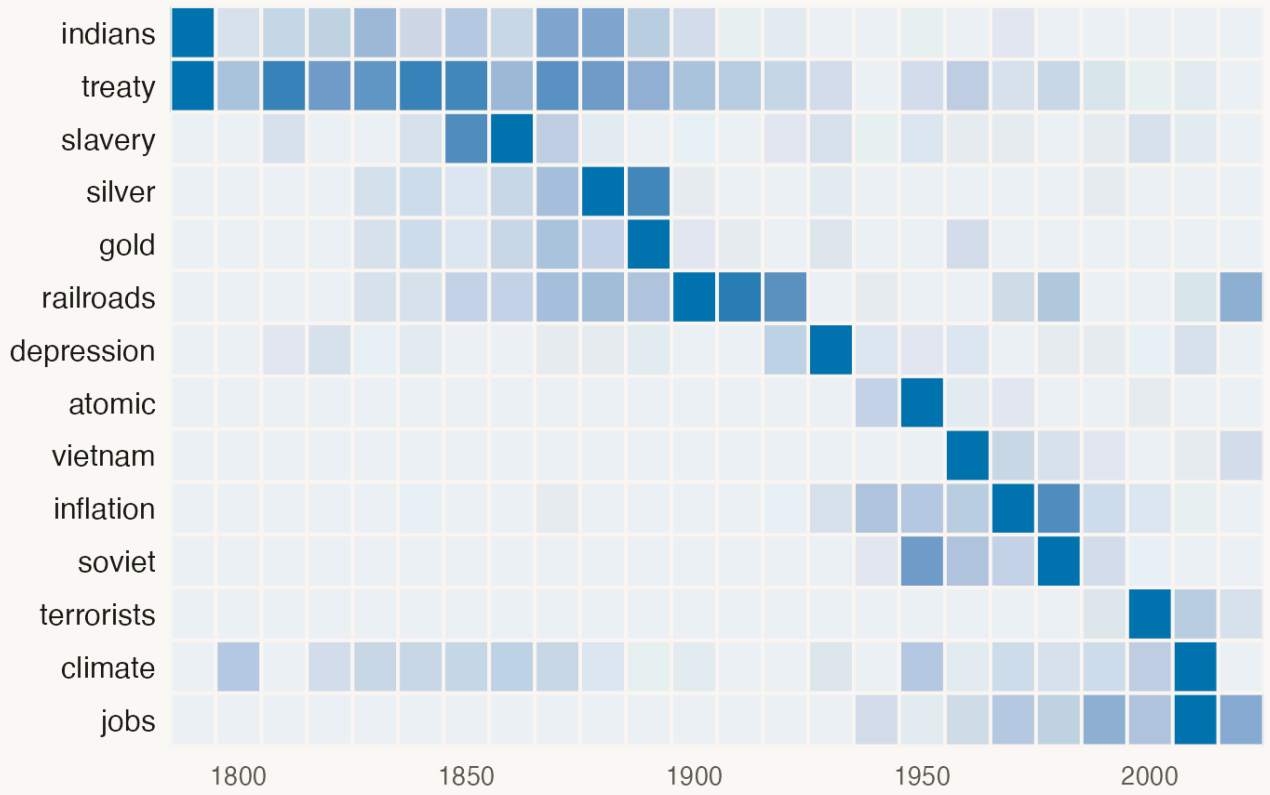


> **Distinctive is not the same as important.**

The vocabulary follows the history

- One row per word, one column per decade of speeches
- The darker the tile, the closer the decade is to the word's own peak
- The diagonal is the history: treaties, slavery, gold, the Soviets, terrorists, jobs

> **Counting alone already shows what each era talked about.**



What a count can tell you, and what it cannot

A COUNT CAN SAY

- Which words a corpus uses, and how unevenly
- Which words set one document apart from the rest
- How the vocabulary shifts over time

A COUNT CANNOT SAY

- Whether a speech is hopeful or fearful
- Whether a tweet is *about* the vaccine, or *against* it
- Anything that needs the researcher's concept rather than the corpus's words

> **To get from words to a concept, somebody has to decide which words the concept consists of. That is a dictionary!**

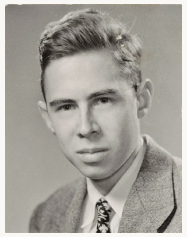
II · Dictionaries

The first automated content analysis

1958

The perceptron

- Frank Rosenblatt's first trainable **neural network**: weighted inputs, a threshold, a learning rule
- Minsky & Papert (1969) show what one layer cannot learn; neural nets go out of fashion



1966

The first dictionary: the General Inquirer

- Stone, Dunphy, Smith & Ogilvie (Harvard): word lists per category, applied to texts by a mainframe — the first computer **content analysis**, and the **dictionary approach** of week 2
- Same year: Weizenbaum's ELIZA imitates a therapist by pattern matching; the ALPAC report ends US funding for machine translation

1970s–80s

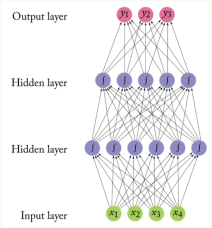
Knowledge-based NLP

- SHRDLU (1970) follows commands in a blocks world; expert systems
- Meaning is encoded by hand in ontologies: concepts, properties, relations
- Works in toy worlds, does not scale to real text

1986

Backpropagation

- Rumelhart, Hinton & Williams: multi-layer networks can be trained
- Neural nets return — but data and compute are still too scarce for language

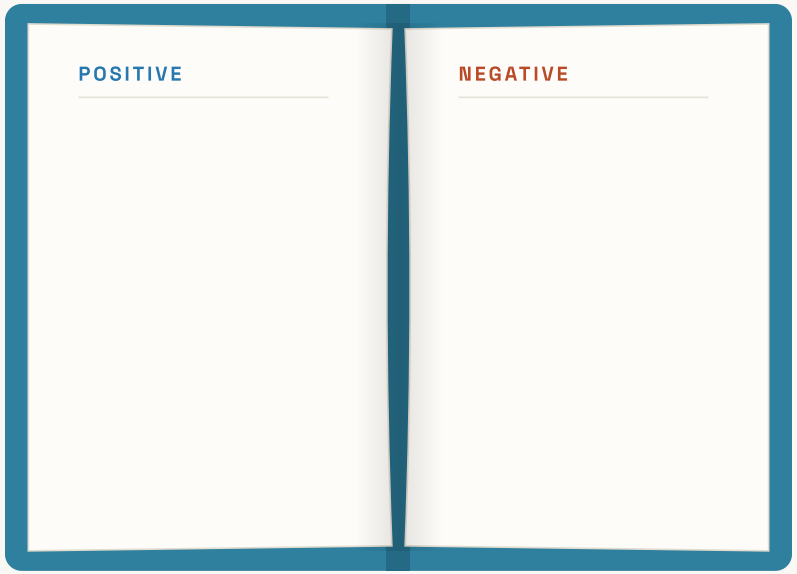


3 / 6

Stone, P. J., Dunphy, D. C., Smith, M. S., & Ogilvie, D. M. (1966). *The General Inquirer: A computer approach to content analysis*. MIT Press.

What a dictionary approach is

- A **dictionary** is a list of words per category: *positive* = {good, safe, hope, ...}; *negative* = {bad, threat, war, ...}
- A **counting rule**: how many words from each list occur in a text, usually per thousand words
- A **decision rule**: a text is *positive* if positive words outnumber negative ones, *about migration* if any migration word occurs
- The book is written **before** the machine sees a text, by the researcher, so the method is deductive



> **"If word X occurs, the text means Y": step 03 of the pipeline, and the simplest classifier this course will use.**

What a dictionary approach is

- A **dictionary** is a list of words per category: *positive* = {good, safe, hope, ...}; *negative* = {bad, threat, war, ...}
- A **counting rule**: how many words from each list occur in a text, usually per thousand words
- A **decision rule**: a text is *positive* if positive words outnumber negative ones, *about migration* if any migration word occurs
- The book is written **before** the machine sees a text, by the researcher, so the method is deductive



> **"If word X occurs, the text means Y": step 03 of the pipeline, and the simplest classifier this course will use.**

What a dictionary approach is

- A **dictionary** is a list of words per category: *positive* = {good, safe, hope, ...}; *negative* = {bad, threat, war, ...}
- A **counting rule**: how many words from each list occur in a text, usually per thousand words
- A **decision rule**: a text is *positive* if positive words outnumber negative ones, *about migration* if any migration word occurs
- The book is written **before** the machine sees a text, by the researcher, so the method is deductive

> **"If word X occurs, the text means Y": step 03 of the pipeline, and the simplest classifier this course will use.**

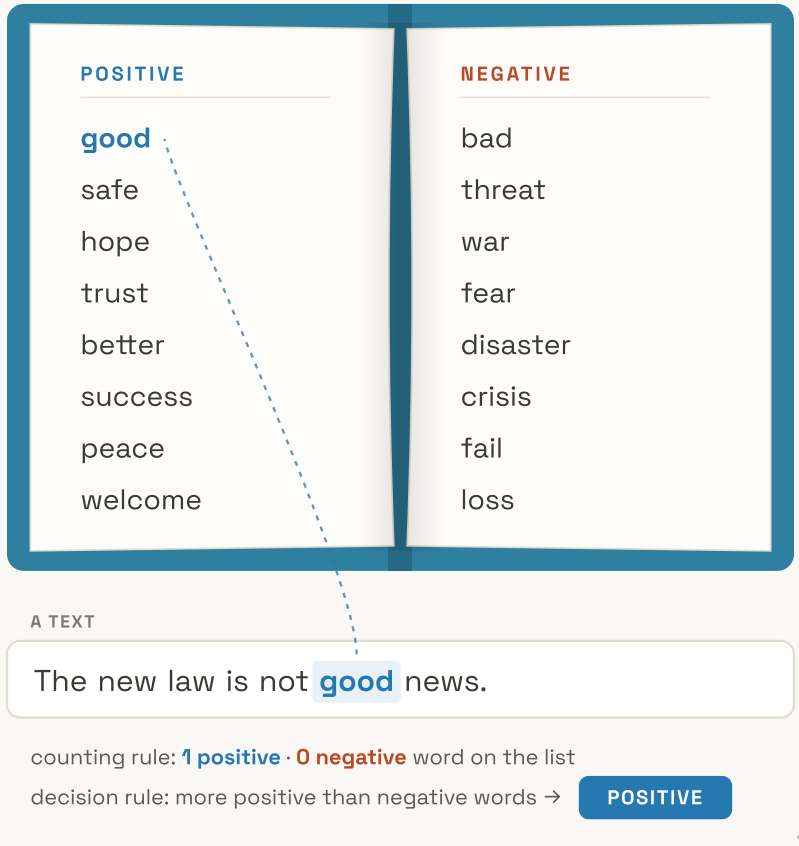


3 / 4

What a dictionary approach is

- A **dictionary** is a list of words per category: *positive* = {good, safe, hope, ...}; *negative* = {bad, threat, war, ...}
- A **counting rule**: how many words from each list occur in a text, usually per thousand words
- A **decision rule**: a text is *positive* if positive words outnumber negative ones, *about migration* if any migration word occurs
- The book is written **before** the machine sees a text, by the researcher, so the method is deductive

> **"If word X occurs, the text means Y": step 03 of the pipeline, and the simplest classifier this course will use.**



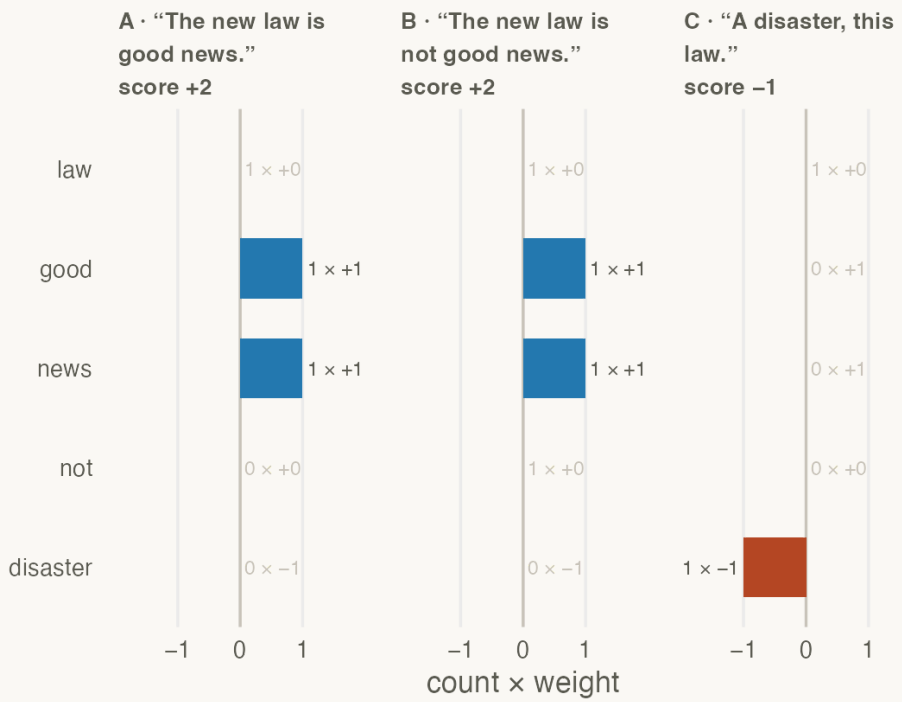
A dictionary is arithmetic on the matrix

```r
# week 1's three sentences, as a document–term matrix
dtm <- texts |>
  unnest_tokens(word, text) |>
  count(doc, word) |>
  pivot_wider(names_from = word, values_from = n, values_fill = 0)

# the dictionary: a weight for every column it knows, 0 otherwise
weights <- c(good = 1, news = 1, disaster = -1)

dtm |>
  mutate(score = good * 1 + news * 1 + disaster * -1) |>
  select(doc, good, news, disaster, score)
```

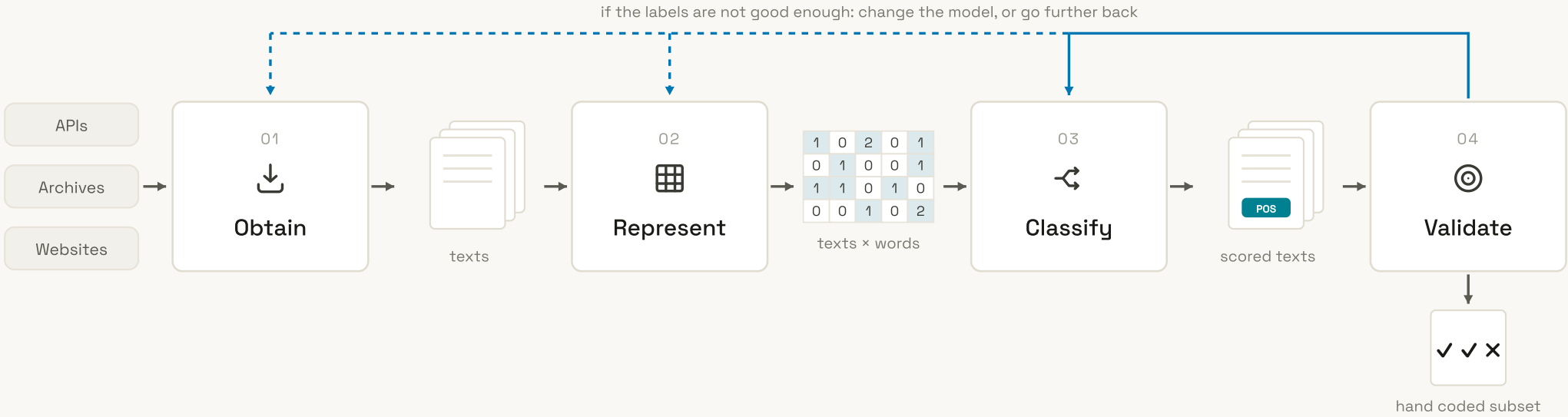
- A word list is a technically just a vector of weights over the matrix's columns: +1, −1, or 0 for the words it does not know
- The score of a text is its row multiplied by that vector: a weighted row sum
- Text B scores +2, like text A. *not* has a column and a zero weight; give it a negative one and *"not a disaster"* is punished just as hard.



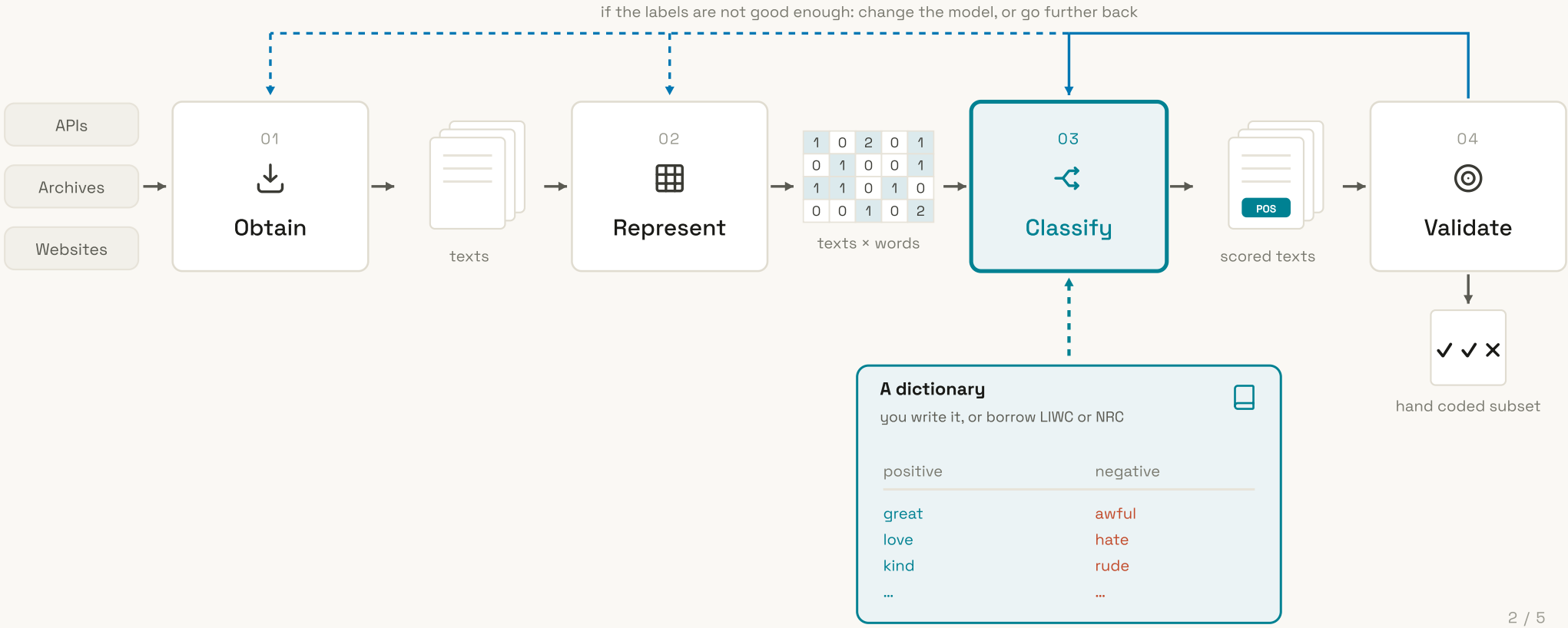
PLAYGROUND Multiply, then add

> Same matrix as last week. The list only decides which columns count, and with what sign.

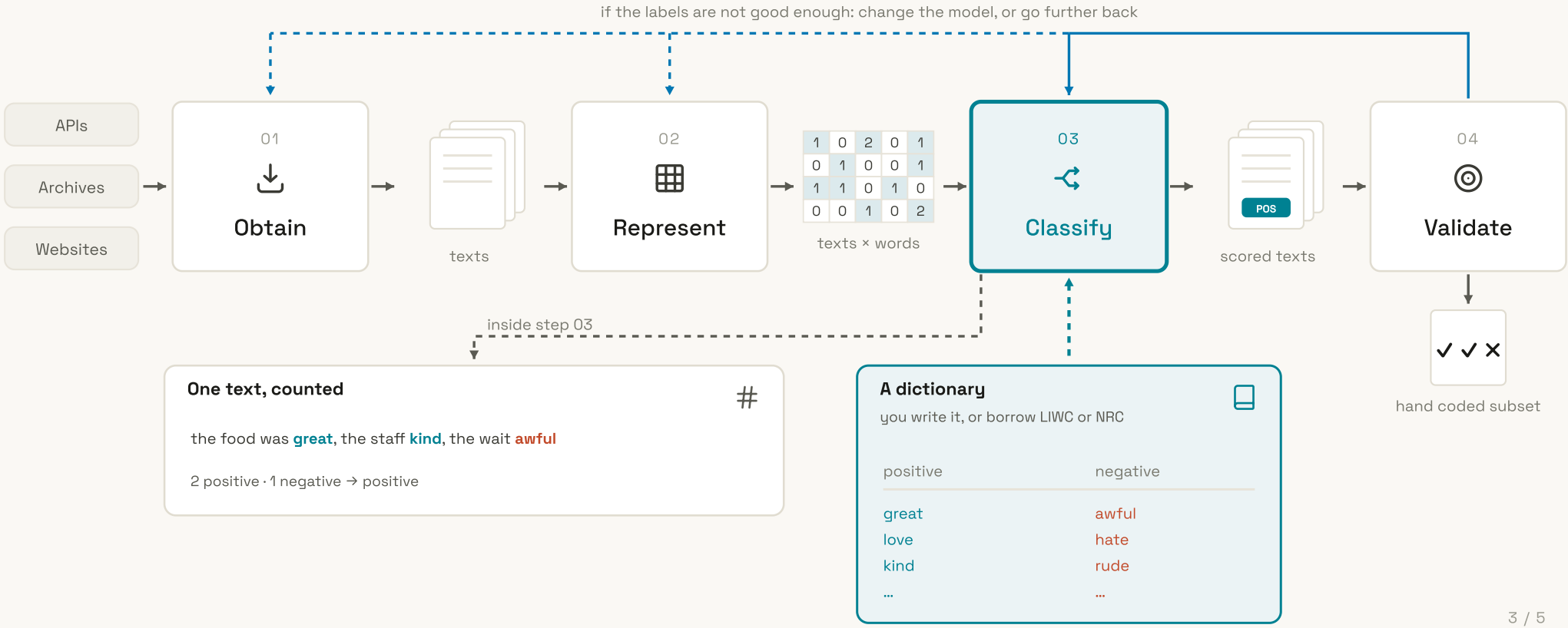
The pipeline with a dictionary



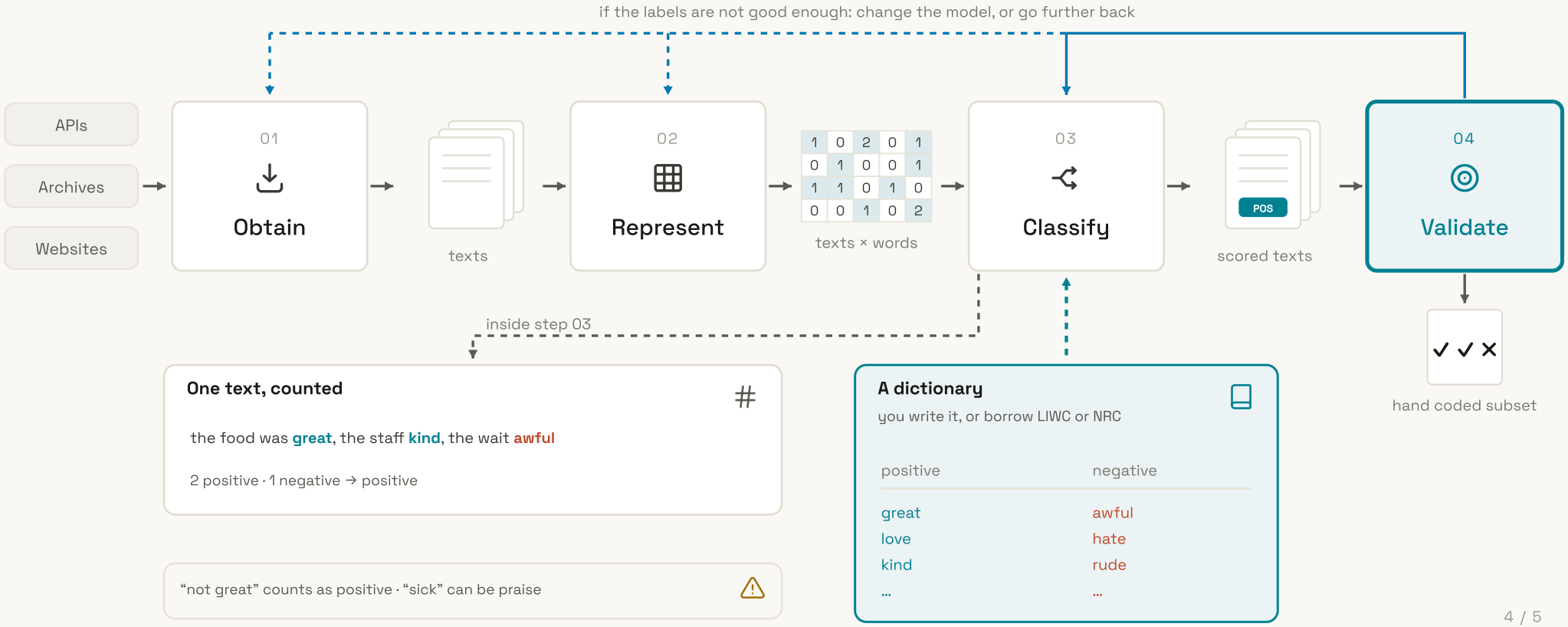
The pipeline with a dictionary



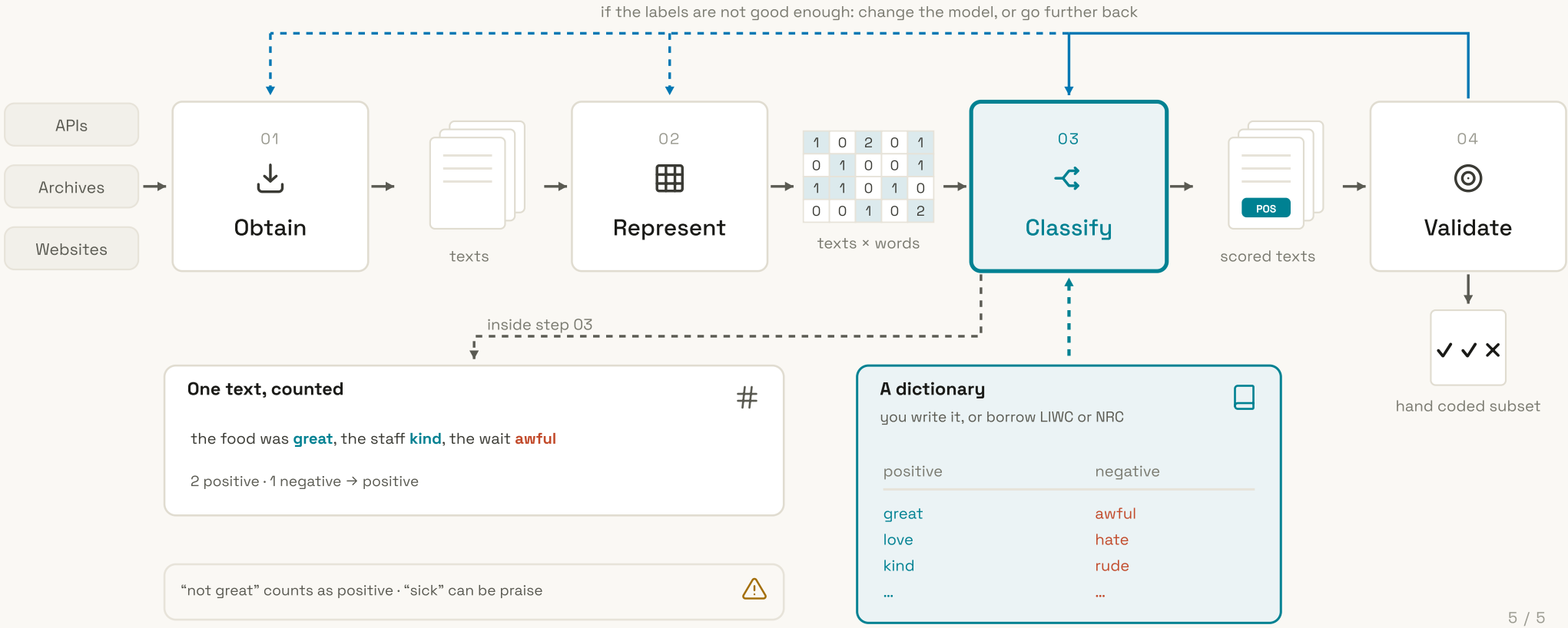
The pipeline with a dictionary



The pipeline with a dictionary



The pipeline with a dictionary



Keyword in context

```
sotu_tokens |>
  group_by(doc_id) |>
  mutate(
    before = str_c(lag(word, 3), lag(word, 2),
                   lag(word, 1), sep = " "),
    after  = str_c(lead(word, 1), lead(word, 2),
                   lead(word, 3), sep = " ")) |>
  ungroup() |>
  filter(str_starts(word, "terror"),
         year %in% c(1850, 1883, 1918, 1987, 2011)) |>
  transmute(year, kwic = str_c(
    before, " [", word, "] ", after))
```

- Before we put a word on a list, we should understand how it is used
- Each hit comes with the three words on either side
- A **keyword-in-context** listing is the cheapest validity check one can run

OUTPUT

```
# A tibble: 5 × 2
   year kwic
  <int> <chr>
1  1850 source of constant [terror] and annoyance to
2  1883 puritan amphitrite and [terror] have been launched
3  1918 from the german [terror] and whom we
4  1987 we yield to [terrorist] blackmail and now
5  2011 the hands of [terrorists] because of a
```

> In 1883 *terror* is a navy monitor, a hit no dictionary should count.

A dictionary is nothing but a table

```r
library(textdata)
nrc <- lexicon_nrc()

nrc |> slice_sample(n = 6)

nrc |> count(sentiment, sort = TRUE)
```

- One row per word–category pair; a word can sit in several categories
- The NRC lexicon as example: about 6,000 English words in eight emotions and two polarities, rated by crowd workers

OUTPUT

```
# A tibble: 6 × 2
  word       sentiment
  <chr>      <chr>
1 caries     disgust
2 abnormal   disgust
3 armed      fear
4 autopsy    sadness
5 elated     joy
6 visitation negative
```

```
# A tibble: 10 × 2
  sentiment     n
  <chr>     <int>
1 negative   3316
2 positive   2308
3 fear       1474
4 anger      1245
# ℹ 6 more rows
```

Mohammad, S. M., & Turney, P. D. (2013). Crowdsourcing a word–emotion association lexicon. *Computational Intelligence, 29*(3), 436–465. https://doi.org/10.1111/j.1467-8640.2012.00460.x

Feature engineering for a word list

```
# tokens as they are
sotu_tokens |>
  inner_join(nrc, by = "word") |> nrow()

# tokens reduced to their lemma first
sotu_tokens |>
  mutate(word = textstem::lemmatize_words(word)) |>
  inner_join(nrc, by = "word") |> nrow()
```

OUTPUT

```
    raw lemmatised
 552320     698513
```

- The list says *nation*; the text says *nations*, this would lead to no match
- Lemmatising the text first finds a quarter more hits!
- Stemming would find fewer: the stem *nation* matches, but *govern* does not match *government* in the list
- Do not remove stop words before a sentiment list: *no*, *not*, *never* are the words you will need in a moment

> **Last week's preprocessing decisions are this week's measurement error.**

The dictionary analysis, step by step · 1

```
sotu_emotions <- sotu_speeches |>
  unnest_tokens(word, text)                # 1 · tokens
```

- Same call as last week: the speech becomes two million rows
- Metadata rides along on every row, so the president and the year are still there at the end

The dictionary analysis, step by step · 2

```
sotu_emotions <- sotu_speeches |>
  unnest_tokens(word, text) |>              # 1 · tokens
  mutate(word = lemmatize_words(word))      # 2 · lemmas
```

- Normalise the text to the form the list uses, not the other way round
- A list of stems would need stemmed text; a list of lemmas needs lemmatised text

The dictionary analysis, step by step · 3

```
1  sotu_emotions <- sotu_speeches |>
2    unnest_tokens(word, text) |>                # 1 · tokens
3    mutate(word = lemmatize_words(word)) |>     # 2 · lemmas
4    inner_join(nrc, by = "word") |>             # 3 · join
5    count(doc_id, president, year, sentiment)   # 4 · count
6
7  sotu_emotions
```

- `inner_join` keeps only the tokens the list knows; `left_join` would keep every token and put `NA` in the category
- The join is the whole classifier. Everything else is bookkeeping

OUTPUT

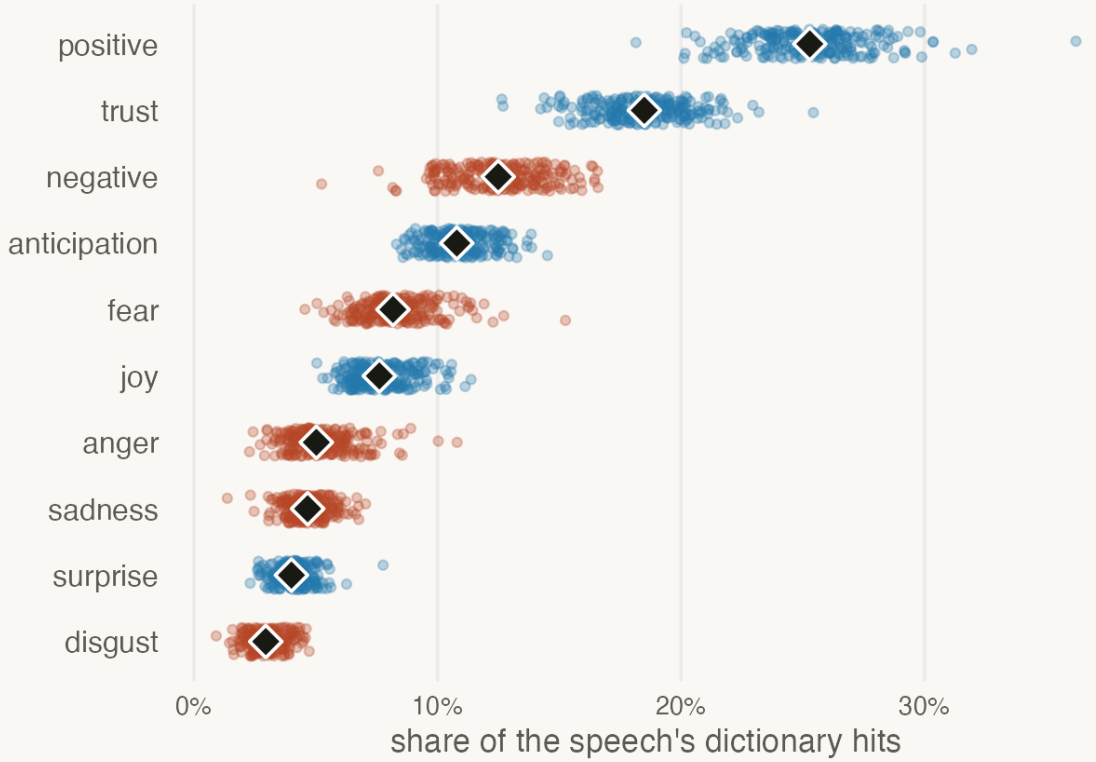
```
# A tibble: 2,400 × 4
  doc_id  year sentiment        n
   <int> <int> <chr>        <int>
1      1  1790 anger           10
2      1  1790 anticipation    55
3      1  1790 disgust          4
4      1  1790 fear            20
5      1  1790 joy             40
6      1  1790 negative        23
7      1  1790 positive       159
8      1  1790 sadness          6
# ℹ 2,392 more rows
```

Emotion in two hundred and forty speeches

```
sotu_emotions |>
  group_by(doc_id) |>
  mutate(share = n / sum(n)) |>
  ggplot(aes(share, sentiment)) +
  geom_jitter(height = .2, alpha = .3) +
  stat_summary(fun = median, geom = "point")
```

- One dot per speech; the marker is the median
- Presidents are trusting and positive by profession; fear and disgust are rare on this stage
- The spread is the interesting part: some speeches are three times as fearful as others

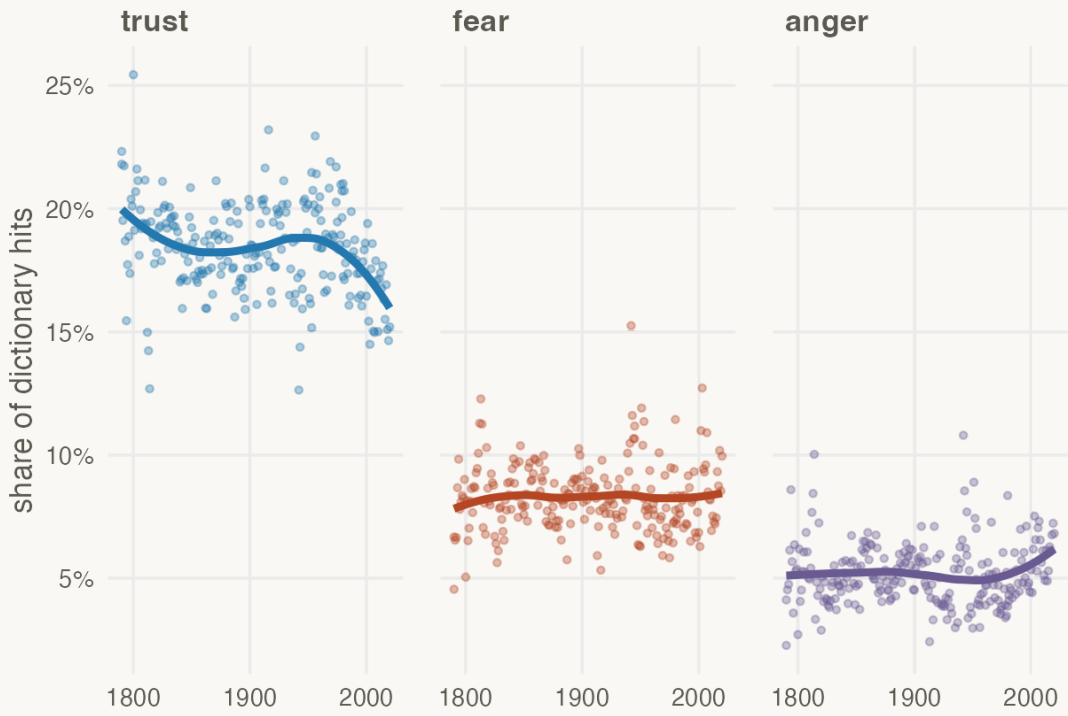
> **A share of matched words, not a share of all words: silence is not counted here.**



Emotion over two centuries

```
sotu_emotions |>
  filter(sentiment %in%
           c("fear", "trust", "anger")) |>
  group_by(doc_id) |>
  mutate(share = n / sum(n)) |>
  ggplot(aes(year, share, colour = sentiment)) +
  geom_point(alpha = .3) +
  geom_smooth(se = FALSE) +
  facet_wrap(~ sentiment, nrow = 1)
```

- Trust words fall away after 1950; anger words rise after 1990; fear is flat, with spikes in 1812, the 1860s and the 1940s
- Each dot is one speech; the smooth line is a loess fit through them



> **Before you believe the trend: what else changed in 230 years?**

What a dictionary score leaves open

- Whether *died* in “three people died in the crash” is a negative **opinion** or a negative **fact**
- Whether the texts with no word on the list are neutral, or just use other words
- Whether “great, another delay” is positive, as the list says, or sarcastic, as you read it
- Whether a list built from product reviews in 2004 or rated in 2011 is about the same thing as your construct

> **A dictionary returns a number for every text. Whether the number measures anything cannot be read off the number.**

Hu, M., & Liu, B. (2004). Mining and summarizing customer reviews. *Proceedings of the 10th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 168–177. https://doi.org/10.1145/1014052.1014073

III · Validation

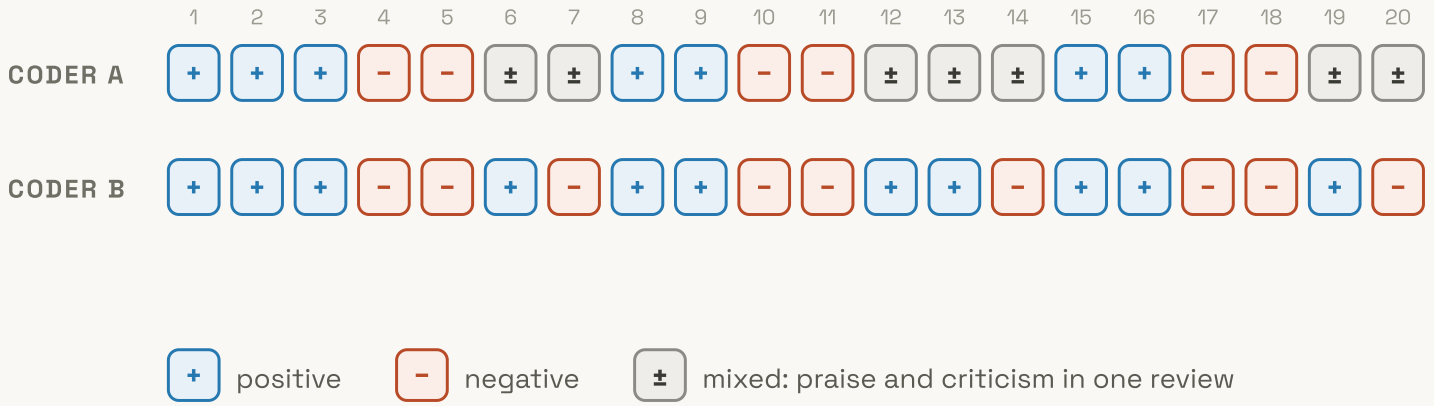
Where a gold standard comes from

1. Draw a sample of texts the instrument will be judged on — random, or stratified so rare cases are in it
2. Write a **codebook**: the categories, and a rule for every case the coders will argue about
3. Two or more people code the sample **independently**, without seeing the instrument's output
4. Measure their **agreement**; where it is poor, the codebook is the problem, not the coders
5. Only then compare the instrument against the human codes

> **Gold is a metaphor. What you have is the best reading humans can agree on, under rules you wrote.**

Krippendorff, K. (2019). *Content analysis: An introduction to its methodology* (4th ed.). Sage. Chapters 11–12.

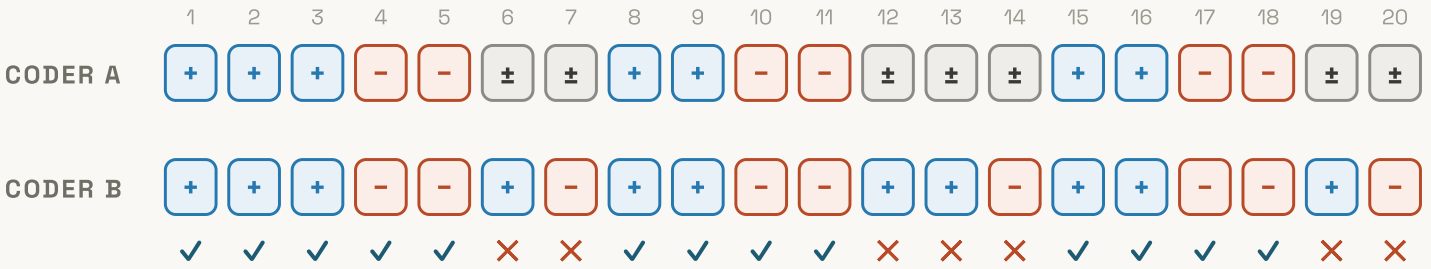
Two coders, twenty reviews



PLAYGROUND Two coders, one codebook

1 / 5

Two coders, twenty reviews



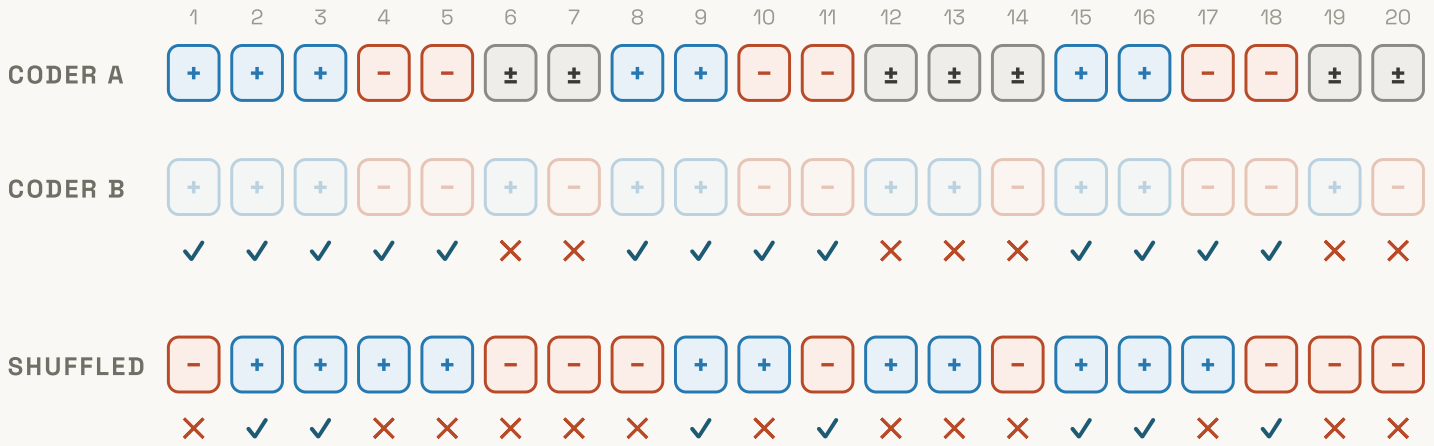
AGREEMENT

65%

13 of 20 reviews match

PLAYGROUND Two coders, one codebook

Two coders, twenty reviews



AGREEMENT

65%

13 of 20 reviews match

BY LUCK

33%

same labels, random order: 7 match here, 6.55 on average

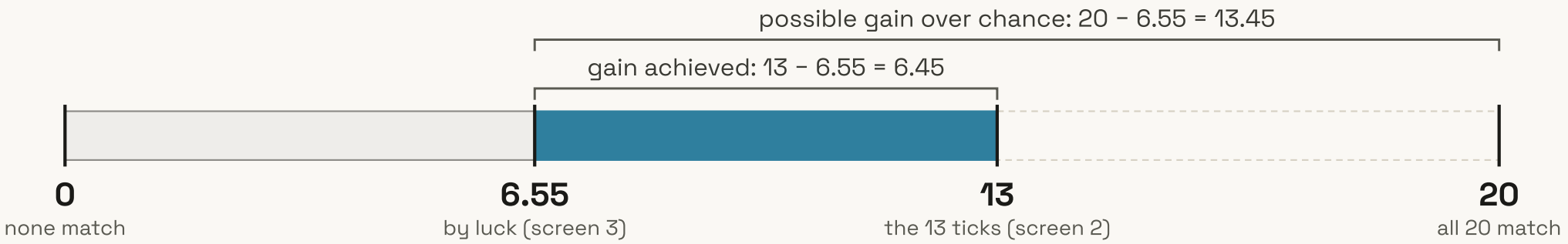
by hand: (7 × 11 + 6 × 9 + 7 × 0) ÷ 20 = 6.55

PLAYGROUND Two coders, one codebook

3 / 5

Two coders, twenty reviews

AGREEMENT, IN REVIEWS OUT OF 20



COHEN'S κ

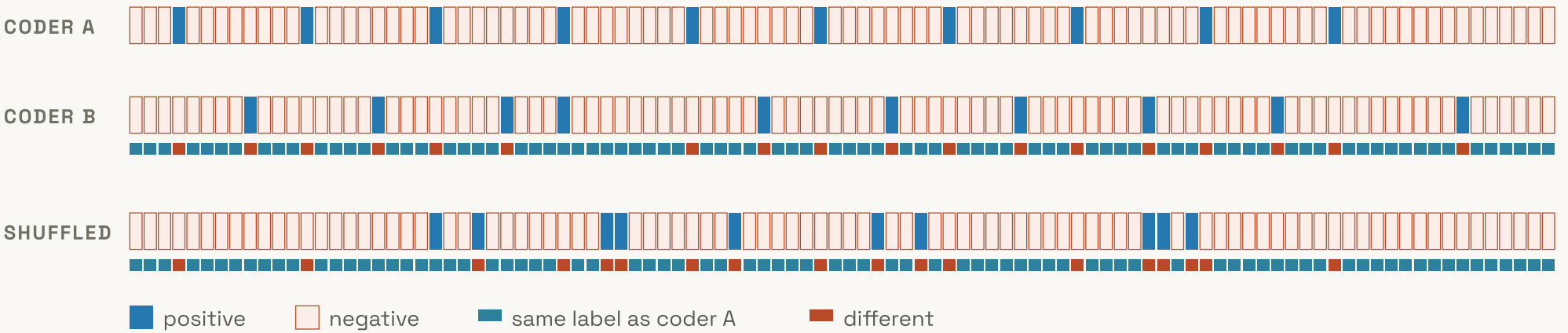
κ = 6.45 ÷ 13.45 = 0.48

How far the coders got from chance (0) to perfect agreement (1).

PLAYGROUND Two coders, one codebook

4 / 5

Two coders, twenty reviews



AGREEMENT

82%

82 of 100 match

BY LUCK

82%

the shuffled row matches on 82

COHEN'S κ

0.00

no better than luck

Nearly every item is negative, so luck alone already matches on most of them.

Cohen's κ

- Agreement between two coders, corrected for the agreement they would reach by luck
- p_o: the share of items both coders label the same, here 13 of 20 = .65
- p_e: the agreement luck gives when each coder keeps their own label shares, here .33
- 1 is perfect agreement, 0 is no better than luck

FORMULA

$\kappa = (p_o - p_e) / (1 - p_e) = (.65 - .33) / (1 - .33) = .48$

```
1  coded <- tibble(item = 1:20, coder_a, coder_b)
2
3  library(yardstick)        # part of tidymodels
4  coded |> kap(coder_a, coder_b)
```

OUTPUT

```
# A tibble: 1 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 kap     multiclass     0.480
```

> The twenty reviews of the explainer: 65% agreement is a κ of .48.

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement, 20*(1), 37–46. https://doi.org/10.1177/001316446002000104

Krippendorff's α

- Disagreement between coders, compared with the disagreement expected by luck
- D_o: the share of items the coders disagree on, here 7 of 20 = .35
- D_e: how often two codes drawn at random from all codes pooled differ, here .64
- Works for any number of coders, with missing codes, and for ordinal or interval scales

FORMULA

$\alpha = 1 - D_o / D_e = 1 - .35 / .64 = .46$

```r
codes <- c(coded$coder_a, coded$coder_b)
n_c <- table(codes)                # 18, 15, 7
N   <- length(codes)               # 40

D_o <- mean(coded$coder_a != coded$coder_b)
D_e <- 1 - sum(n_c * (n_c - 1)) / (N * (N - 1))
round(1 - D_o / D_e, 2)
```

OUTPUT

```
[1] 0.46
```

> .46 against κ's .48. They differ because only one coder used *mixed*.

Krippendorff, K. (2011). *Computing Krippendorff's alpha-reliability*. University of Pennsylvania. https://repository.upenn.edu/asc_papers/43

κ and α side by side

	COHEN'S K	KRIPPENDORFF'S A
Coders	exactly two	any number
Missing codes	not allowed	handled
Level of measurement	nominal (weighted κ for ordinal)	nominal, ordinal, interval, ratio
Chance model	each coder's own marginals	all values pooled
Conventions	none the author endorses; the .41 / .61 / .81 bands are folklore	≥ .80 reportable; ≥ .667 for tentative conclusions
In this course	the lab, because you are two	the group project, because α is what journals ask for

> **Report the number, the number of coders, the number of units, and the category frequencies. Without the last, no coefficient can be read.**

Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics, 33*(1), 159–174. https://doi.org/10.2307/2529310

Agreement is not correctness

- Two coders can agree perfectly and both be wrong: agreement measures shared reading, not truth
- The codebook decides what the gold standard is: a different rule for "praise *and* criticism" produces a different gold set from the same texts
- Every instrument validated against that set inherits the rule, silently
- So the codebook is part of the measurement, and it goes in the methods section

> **A gold standard is something people made. Reliability tells you they made it consistently; nothing tells you they made it right.**

Now the machine, against a thousand labelled sentences

```
1 yelp <- read_csv("data/yelp-sentences.csv")
2
3 yelp |> slice_sample(n = 6)
4
5 yelp |> count(label)
```

- One thousand single-sentence restaurant reviews, each with a human label, half positive and half negative
- The labels play the role of the gold standard; the Bing word list plays the instrument

OUTPUT

```
# A tibble: 6 × 2
  text                                label
  <chr>                               <chr>
1 I love the fact that everything... positive
2 I don't think I'll be running b... negative
3 I can take a little bad service... negative
4 Third, the cheese on my friend'... negative
5 An absolute must visit!           positive
6 I work in the hospitality indus... negative
```

```
# A tibble: 2 × 2
  label        n
  <chr>    <int>
1 negative   500
2 positive   500
```

Kotzias, D., Denil, M., de Freitas, N., & Smyth, P. (2015). From group to individual labels using deep features. *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 597–606. https://doi.org/10.1145/2783258.2783380

Scoring the sentences · 1 · a score per sentence

```
1  scored <- yelp |>
2    unnest_tokens(word, text) |>          # 1
3    inner_join(bing, by = "word") |>      # 2
4    count(id, sentiment) |>               # 3
5    pivot_wider(names_from = sentiment,   # 4
6                values_from = n,
7                values_fill = 0) |>
8    mutate(score = positive - negative)   # 5
9
10 print(scored, n = 5)
```

1. One row per word of each sentence
2. Keep only the words on the Bing list, each with its sentiment
3. Count the positive and the negative words per sentence
4. Back to one row per sentence, with 0 where a sentiment is missing
5. The score: positive words minus negative words

OUTPUT

```
# A tibble: 782 × 4
     id positive negative score
  <dbl>    <int>    <int> <int>
1     1        2        0     2
2     2        1        0     1
3     3        0        1    -1
4     4        2        0     2
5     5        1        0     1
# ℹ 777 more rows
```

> 782 of the 1,000 sentences contain a word on the list. The other 218 are not in `scored` at all.

Scoring the sentences · 2 · from score to verdict

```
verdicts <- yelp |>
  left_join(scored, by = "id") |>     # 1
  mutate(verdict = case_when(         # 2
    score > 0 ~ "positive",
    score < 0 ~ "negative",
    TRUE      ~ "no call"))           # 3

verdicts |>
  filter(id %in% c(1, 2, 12, 18, 32)) |>
  select(text, score, verdict)

count(verdicts, verdict)
```

1. Back to all 1,000 sentences: the 218 without a listed word get `NA` as score
2. The decision rule: above 0 is positive, below 0 is negative
3. Everything else is “no call”: 60 ties and the 218 with `NA`

OUTPUT

```
# A tibble: 5 × 3
  text                                      score verdict
  <chr>                                     <int> <chr>
1 Wow... Loved this place.                      2 positive
2 Crust is not good.                            1 positive
3 Would not go back.                           NA no call
4 Waitress was a little slow in service.       -1 negative
5 This was like the final blow!                 0 no call


# A tibble: 3 × 2
  verdict      n
  <chr>    <int>
1 negative   224
2 no call    278
3 positive   498
```

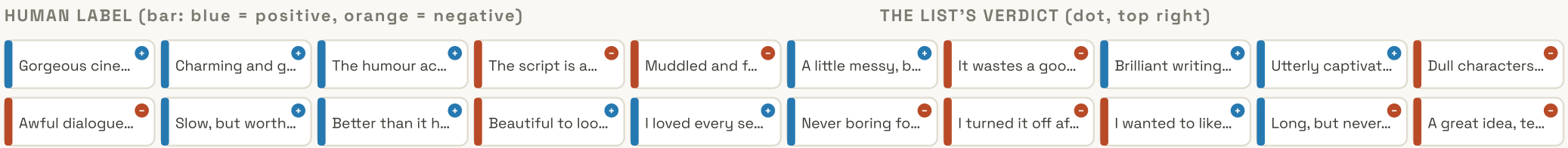
> Every sentence now has two labels: the human’s and the list’s. Validation is the comparison.

The confusion matrix

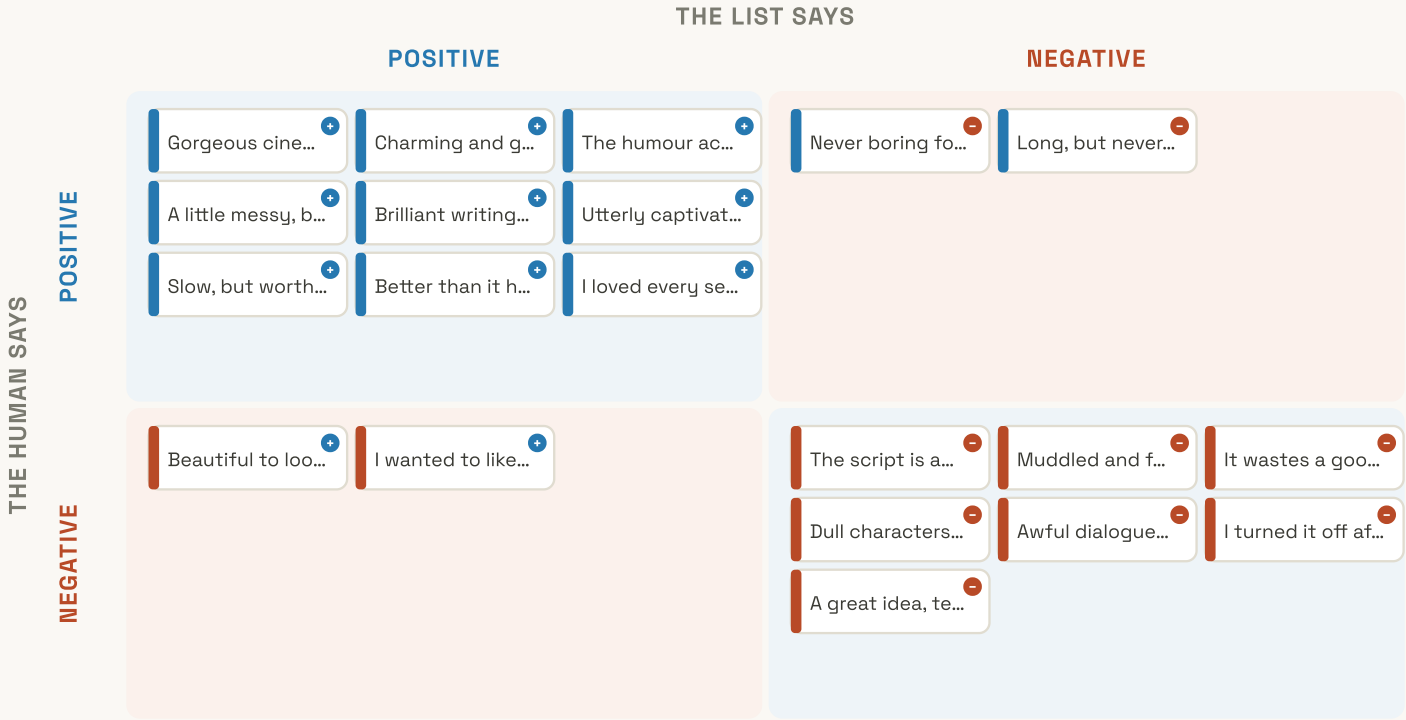
HUMAN LABEL (bar: blue = positive, orange = negative)

Gorgeous cine…	Charming and g…	The humour ac…	The script is a…	Muddled and f…	A little messy, b…	It wastes a goo…	Brilliant writing…	Utterly captivat…	Dull characters…
Awful dialogue…	Slow, but worth…	Better than it h…	Beautiful to loo…	I loved every se…	Never boring fo…	I turned it off af…	I wanted to like…	Long, but never…	A great idea, te…

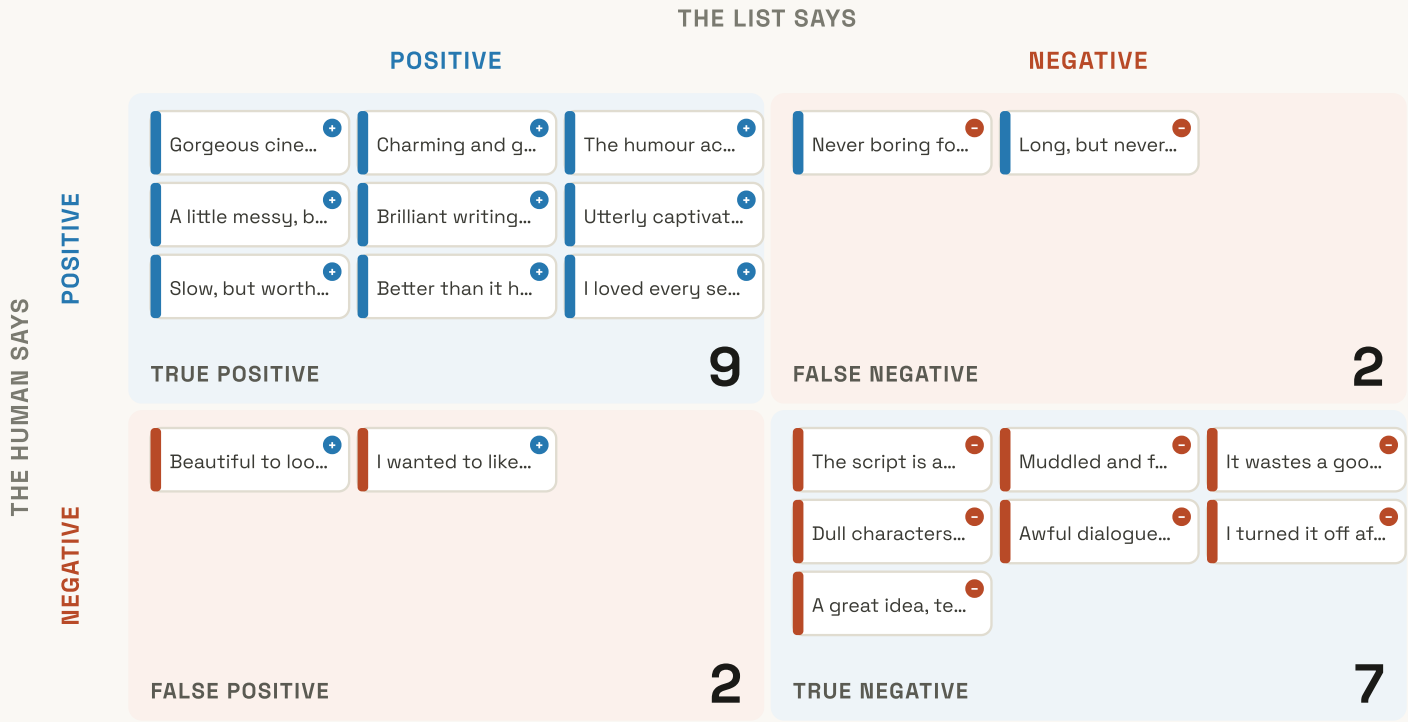
The confusion matrix



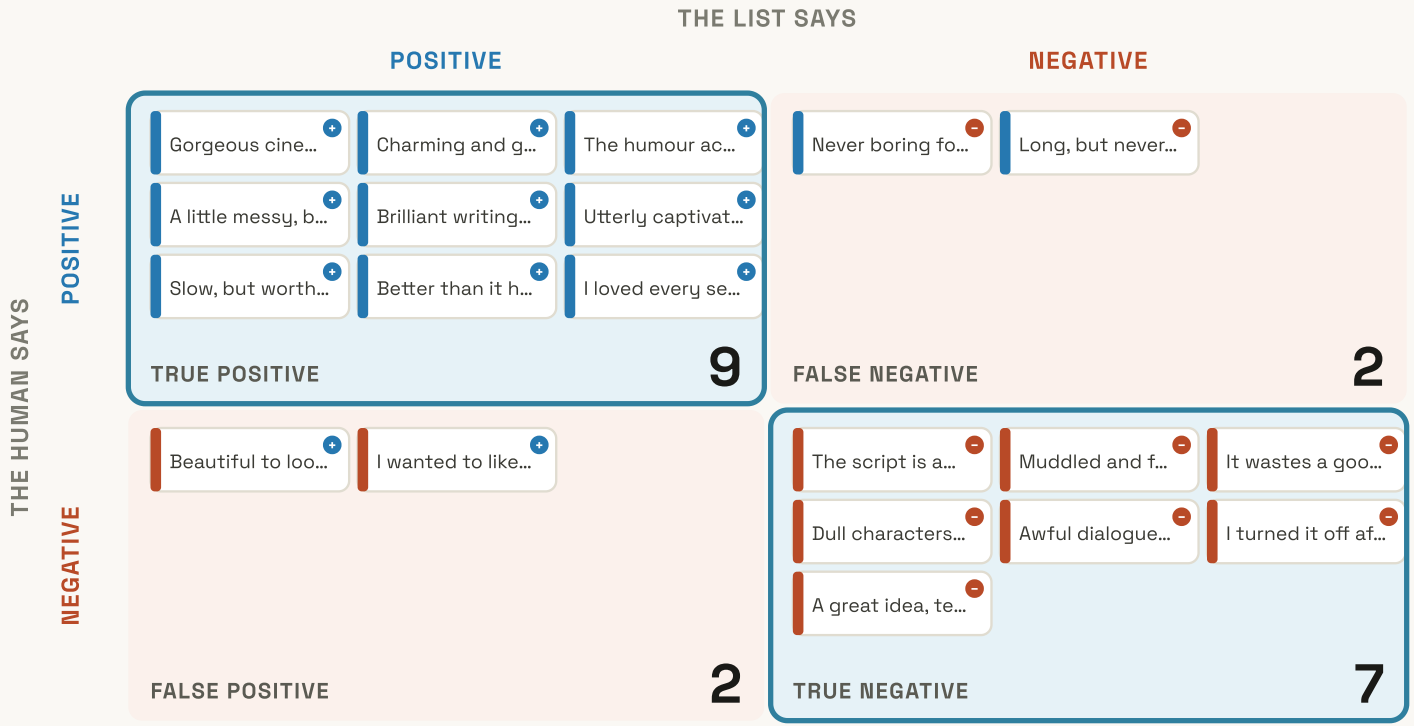
The confusion matrix



The confusion matrix



The confusion matrix



accuracy

$$\frac{TP + TN}{TP + FP + FN + TN}$$

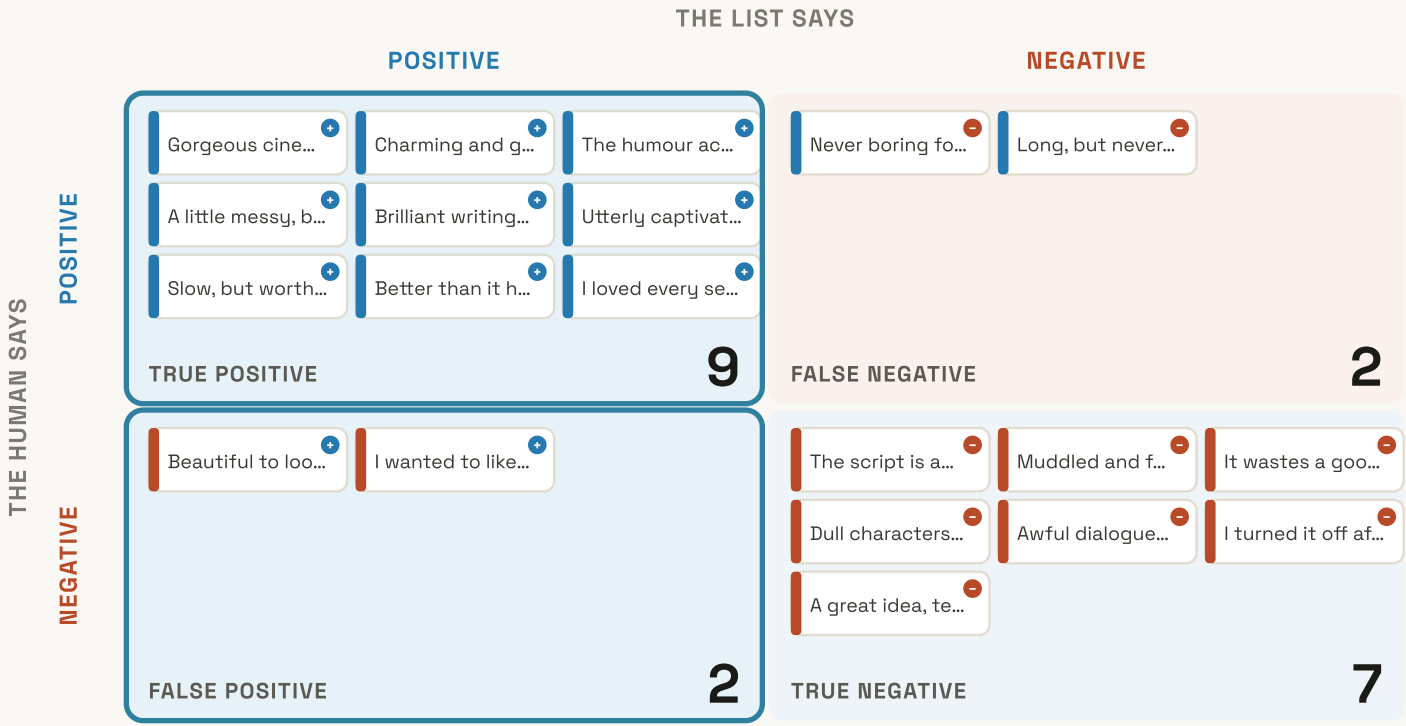
= 9 + 7 / (9 + 2 + 2 + 7)

= 0.80

of all twenty verdicts, the share that were right

5 / 8

The confusion matrix



precision

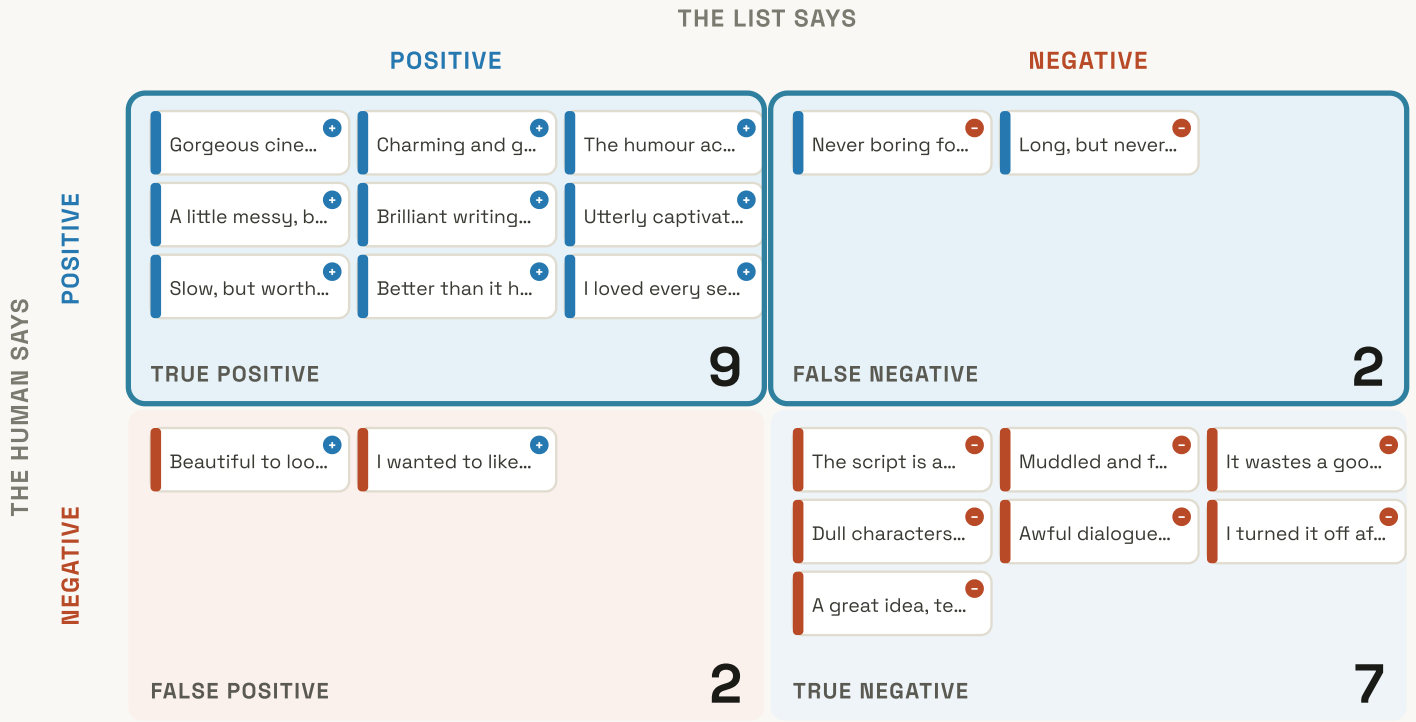
$$\frac{TP}{TP + FP}$$

$= 9 / (9 + 2)$

$= 0.82$

of the reviews the list called positive, the share that were

The confusion matrix



recall

$$\frac{TP}{TP + FN}$$

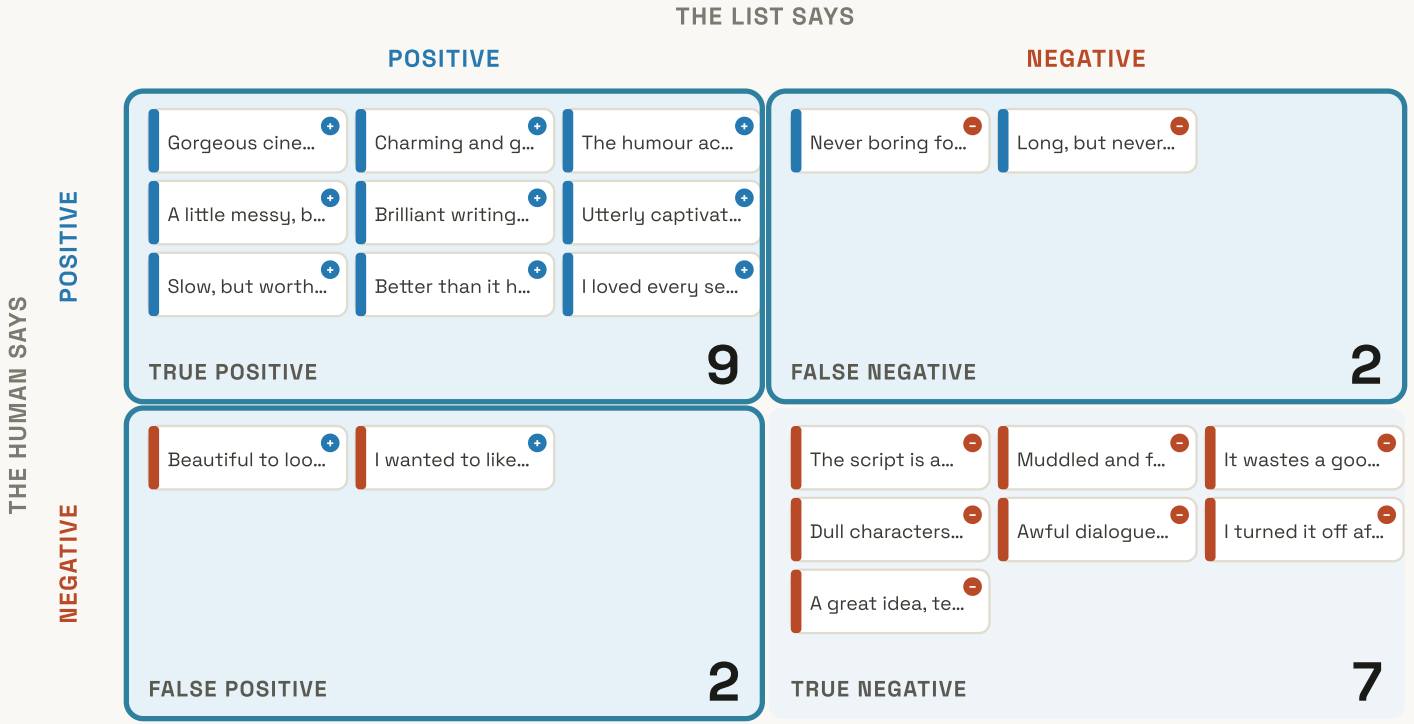
= 9 / (9 + 2)

= 0.82

of the reviews that were positive, the share the list found

7 / 8

The confusion matrix



F1

$$\frac{2 \cdot \mathbf{TP}}{2 \cdot \mathbf{TP} + \mathbf{FP} + \mathbf{FN}}$$

$= 2 \cdot 9 / (2 \cdot 9 + 2 + 2)$

$= 0.82$

the harmonic mean of precision and recall

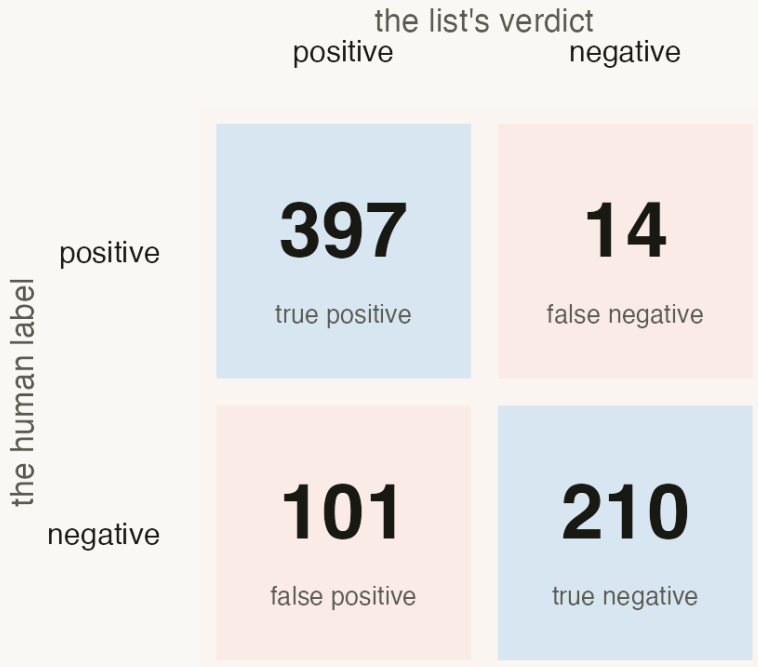
8 / 8

The confusion matrix, on the real data

```r
called <- verdicts |>
  filter(verdict != "no call") |>
  mutate(across(c(label, verdict), factor,
                levels = c("positive", "negative")))

with(called, table(human = label, list = verdict))
```

- 722 sentences the list was willing to judge
- **397** positive sentences it called positive; **210** negative ones it called negative
- **101** negative sentences called positive: the cell with the negations in it
- Only **14** positive sentences missed



> **Rows are the truth, columns the verdict. Every number you report from now on is a ratio of these four cells.**

Accuracy, precision, recall, F1

```r
class_metrics <- metric_set(accuracy, precision,
                            recall, f_meas, kap)

called |>
  class_metrics(truth = label, estimate = verdict)
```

OUTPUT

```
# A tibble: 5 × 2
  .metric   .estimate
  <chr>         <dbl>
1 accuracy      0.841
2 precision     0.797
3 recall        0.966
4 f_meas        0.873
5 kap           0.664
```

- **Accuracy**: share of all verdicts that were right: (TP + TN) / all
- **Precision**: of the sentences called positive, the share that were: TP / (TP + FP)
- **Recall**: of the positive sentences, the share the list found: TP / (TP + FN)
- **F1**: the harmonic mean of the two, which punishes a lopsided pair
- **κ**: accuracy corrected for chance, exactly as between two coders

PLAYGROUND **Build-a-dictionary**

> **Recall .97 and precision .80: the list finds nearly every positive sentence, and calls a fifth of the negative ones positive too.**

Which number do you report?

- A moderation filter for hate speech wants **recall**: a missed case does harm; a wrongly flagged one costs a human review
- A sample of “tweets about the vaccine” for a study wants **precision**: every false positive pollutes the sample
- **F1** when the two errors cost about the same and you need one number to compare instruments
- **Accuracy** only with the base rate beside it; the next slides show why
- All of them **per class**: a positive-class F1 says nothing about the negative class

> **The right metric follows from what an error costs in your study. Decide that in the specification, not after seeing the results.**

Sentences the list cannot call

```
no_call <- verdicts |>
  filter(verdict == "no call") |>
  mutate(why = if_else(is.na(score),
                       "no listed word", "tie"))

count(no_call, why, label)
```

- **No call** means the list has nothing to say: none of the words is on the list, or the positive and negative words cancel out
- *"Would not go back."* has no listed word; *"This was like the final blow!"* is a tie, *like* against *blow*

OUTPUT

```
# A tibble: 4 × 3
  why            label        n
  <chr>          <chr>    <int>
1 no listed word negative   144
2 no listed word positive    74
3 tie            negative    45
4 tie            positive    15
```

- 278 of 1,000 sentences: 218 without a listed word, 60 ties
- They are not neutral: the human labels say 189 of them are negative and 89 positive

> **No call is still an outcome. Before you report a number, decide what happens to these 278 sentences.**

Three ways to count the no-call sentences

RULE	SENTENCES	PRECISION	RECALL	F1
Leave them out	722	0.80	0.97	0.87
Count as negative	1,000	0.80	0.79	0.80
Count as positive	1,000	0.63	0.97	0.76

> **Coverage, the share of texts an instrument says anything about, is a validity number too. Report it.**

- **Leave them out**: the list is judged only where it found words, which flatters it
- **Count as negative**: the 89 positive reviews among them become misses, so recall falls
- **Count as positive**: the 189 negative ones become false alarms, so precision falls
- Negative is the usual default, because the question is whether the list finds the positive reviews, and saying nothing does not find them. It remains a decision, and the paper has to state it

Where it goes wrong: negation

```
verdicts |>
  filter(str_detect(text, "\\bnot\\b|n't"),
         verdict == "positive", label == "negative") |>
  select(text)
```

- The word list has no entry for *not*: *good* scores +1 whether or not it is negated, and the matrix cannot say what the *not* was attached to
- Fifty-eight of the 101 false positives contain *not* or *n't*
- Fixes exist (flip the sign of the next word, count bigrams, use a list with negated entries), and each is a new decision to validate

OUTPUT

```
# A tibble: 8 × 1
  text
  <chr>
1 Crust is not good.
2 Honeslty it didn't taste THAT fresh.)
3 This place is not worth your time, let alone Vegas.
4 did not like at all.
5 There is not a deal good enough that would drag m...
6 It was not good.
7 REAL sushi lovers, let's be honest — Yama is not ...
8 The waiter wasn't helpful or friendly and rarely ...
```

> **"Not good at all" from the Playground, a thousand times over.**

More than two classes: macro and micro

```r
with(three, table(human = gold, list = verdict))

per_class <- three |>
  group_by(verdict) |>
  summarise(precision = mean(gold == verdict))
per_class

round(c(macro = mean(per_class$precision),
        micro = mean(three$gold == three$verdict)), 2)
```

- Per class: of the reviews the list called *positive*, all were right (1.00); of those it called *mixed*, .65
- **Macro** averages the three precisions: every class counts equally
- **Micro** pools all reviews first: every review counts equally, and for precision that is simply accuracy

OUTPUT

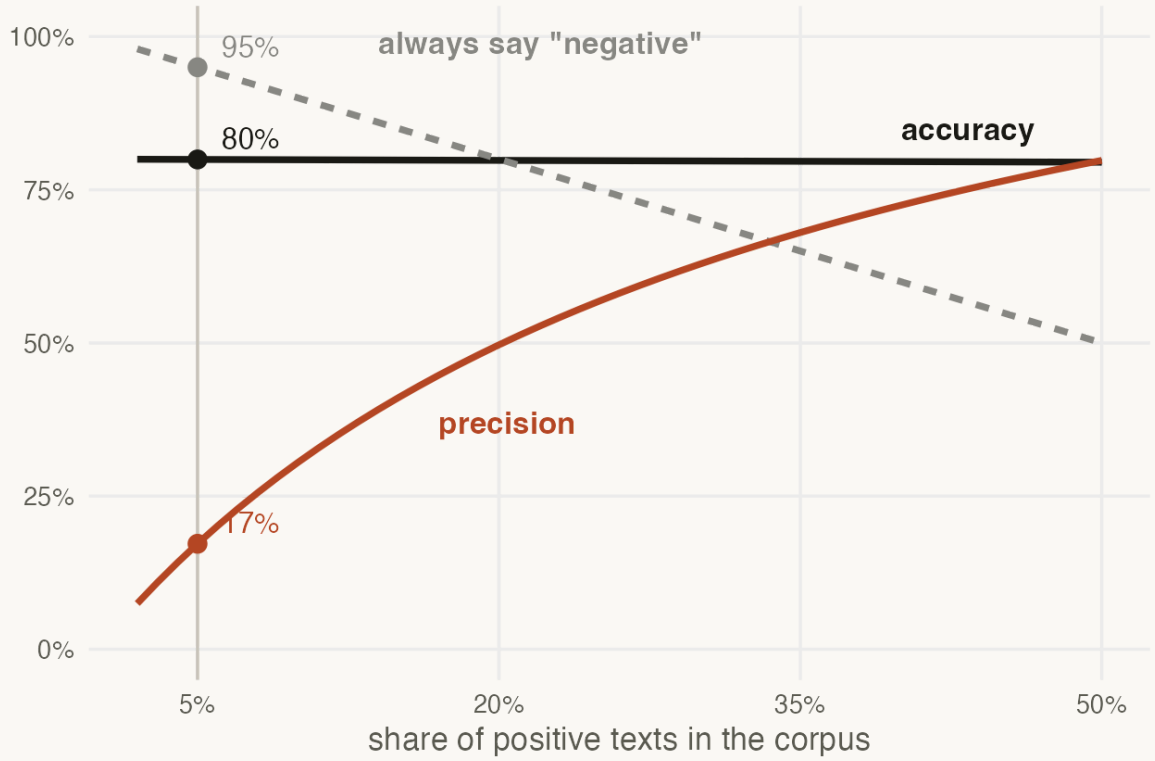
```
          list
human      positive negative mixed
  positive       11        1     3
  negative        0       14     4
  mixed           0        2    13
```

```
# A tibble: 3 × 2
  verdict  precision
  <fct>        <dbl>
1 positive     1
2 negative     0.824
3 mixed        0.65
```

```
macro micro
 0.82  0.79
```

> Report per class, and say which average you used. A single macro figure hides the class your instrument cannot see.

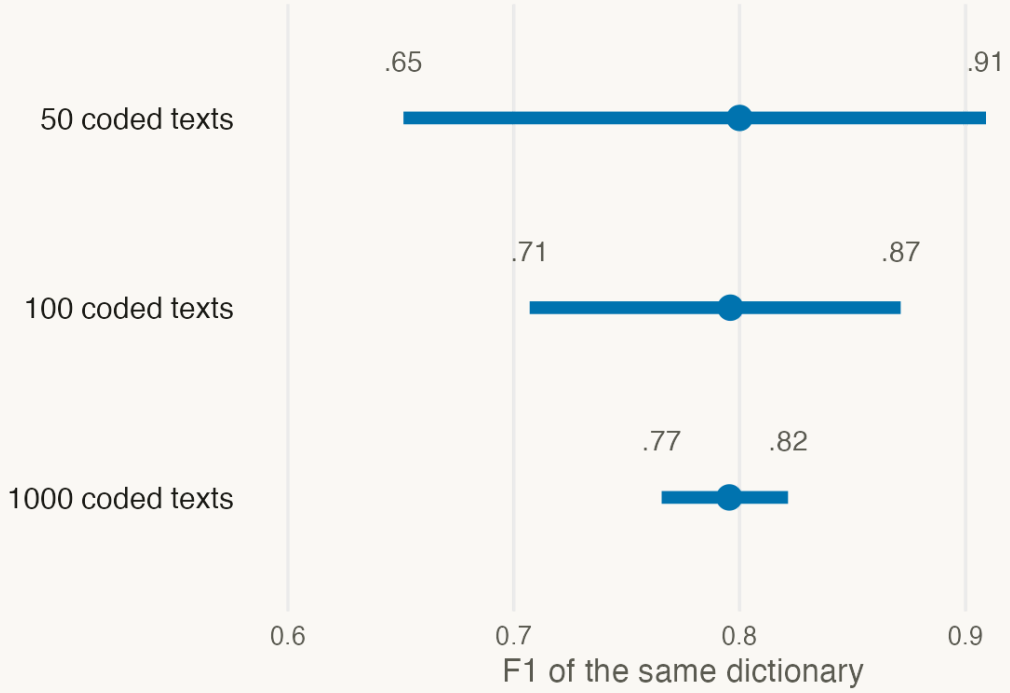
Class imbalance



- The same dictionary, used on corpora where positive texts are more or less common
- Its accuracy stays at 80% everywhere, so it looks equally good in all of them
- When positive texts are rare, most of what it flags is wrong: at 5% positives, only 17% of the flagged texts are positive
- A useless rule, “always say negative”, would even beat it on accuracy

> **When a class is rare, accuracy says almost nothing. Look at precision and recall.**

How sure are you?



Bars: 95% bootstrap confidence intervals, from 1,000 resamples of each size drawn from the 1,000 coded sentences. Dots: median F1.

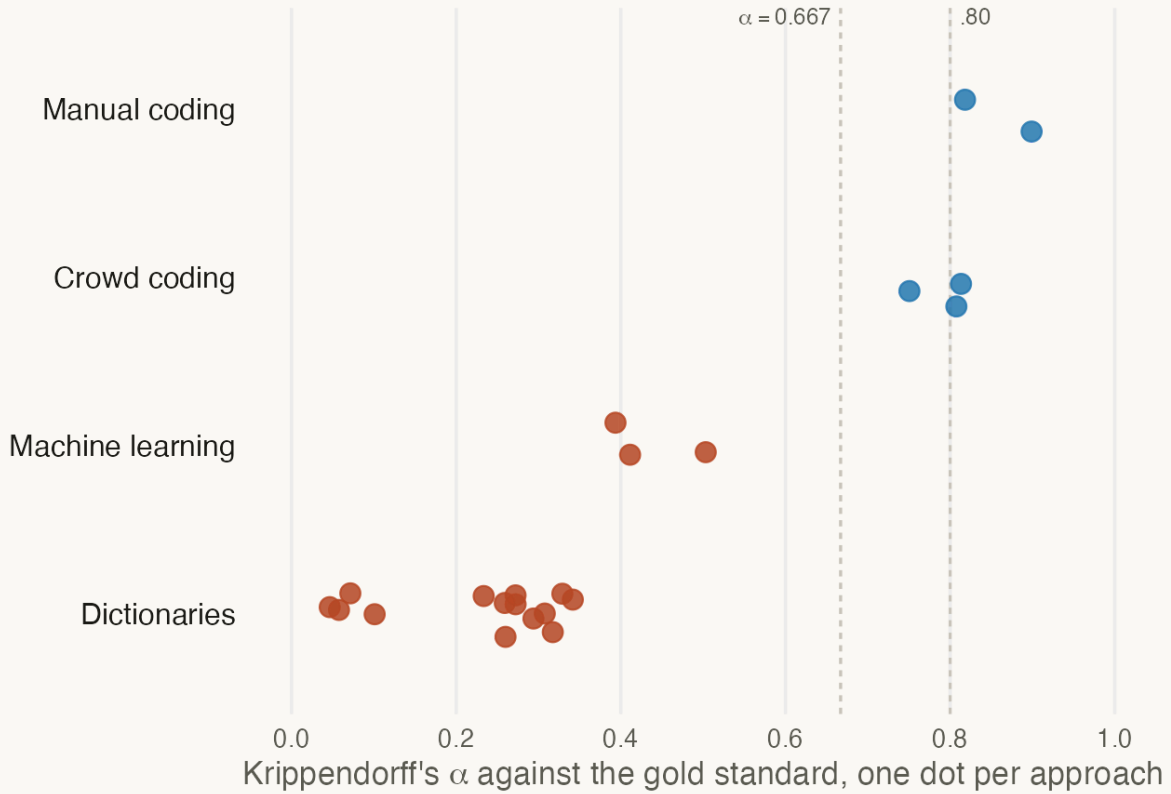
- The same dictionary, checked against hand-coded samples of different sizes
- With 50 coded texts, its F1 could be anywhere between .65 and .91: from unusable to good
- With 1,000 coded texts, it is pinned between .77 and .82
- The smaller the validation sample, the less a validation score tells you

> **A validation set of 50 is a rumour about your instrument. Code more, or report the interval.**

Do dictionaries measure sentiment?

- Dutch economic news, 2015; three-class sentiment, hand-coded gold standard
- The same texts measured by trained coders, crowd workers, three classifiers, and fourteen dictionaries
- Agreement with the gold standard as Krippendorff's α, the way you would report your own coders

> **No dictionary reaches α = .35. The worst human reaches .75. That is the gap this week is about.**



Redrawn from Table 2. Blue: humans, orange-red: machines.

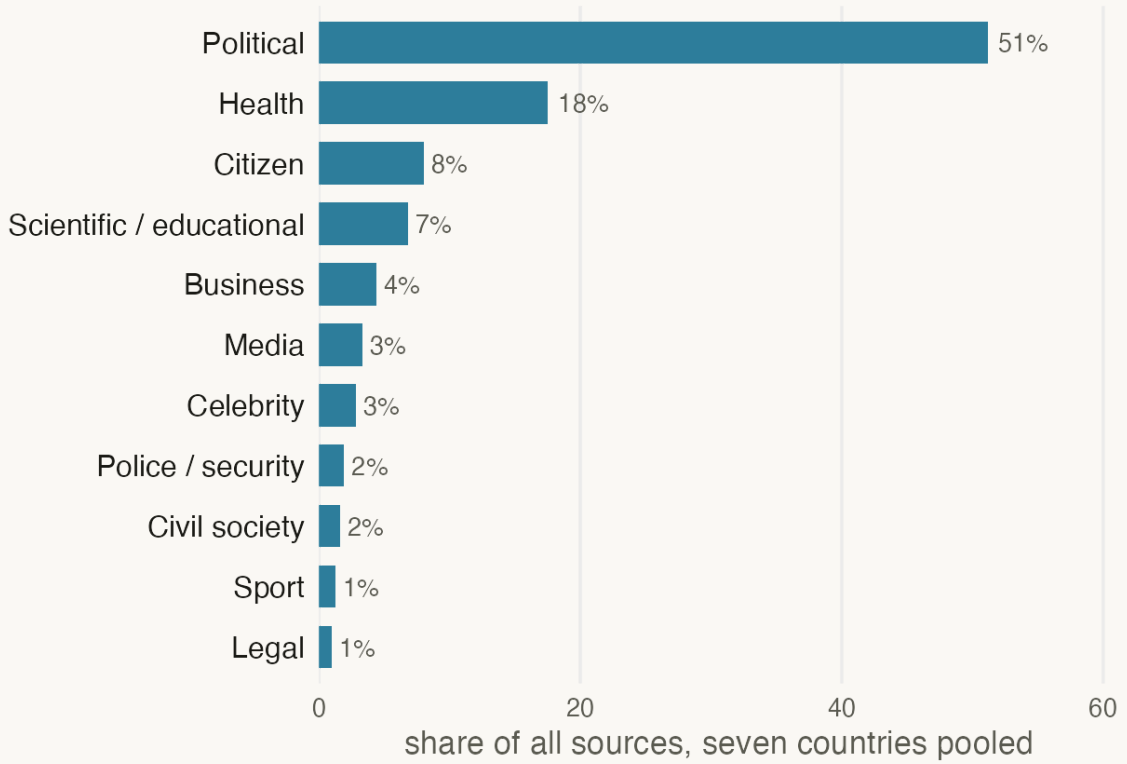
van Atteveldt, W., van der Velden, M. A. C. G., & Boukes, M. (2021). The validity of sentiment analysis: Comparing manual annotation, crowd-coding, dictionary approaches, and machine learning algorithms. *Communication Methods and Measures, 15*(2), 121–140. https://doi.org/10.1080/19312458.2020.1869198

IV · Dictionaries in research

Example 1 · Who gets to speak in pandemic news

- 940,000 posts about COVID-19 from 78 news outlets on Facebook, Instagram and Twitter, seven countries, all of 2020
- Question: which **sources** does pandemic news quote — politicians, doctors, scientists, citizens?
- Half of all sources are political, in every country; health sources second, citizens a distant third
- Elite orientation survives a health emergency

> **A construct that *is* a word list: a source is a name, a title, an institution.**



Redrawn from Table 2 (global column). Mellado et al. (2021).

Mellado, C., Hallin, D., Cárcamo, L., Alfaro, R., Jackson, D., Humanes, M. L., … Ramos, A. (2021). Sourcing pandemic news: A cross-national computational analysis of mainstream media coverage of COVID-19 on Facebook, Twitter, and Instagram. *Digital Journalism, 9*(9), 1271–1295. https://doi.org/10.1080/21670811.2021.1942114

Example 1 · How the dictionary was built

- Eleven source categories, each broken into positions, names, institutions, organisations, nicknames and acronyms
- Each national team translated and extended the list for its own country and language
- **10,102 entries**, a dictionary of *who* is speaking
- Built from the corpus and the coders' knowledge, then applied to all 940,000 posts

> **This is where dictionaries are strongest: a closed, enumerable category, where the presence of a name is the measurement.**

Example 2 · Migration on politicians' Facebook pages

- 24,578 migration-related posts by 1,702 members of parliament in six European countries, 2013–2017
- Two questions: how **visible** is migration on each party's page, and how **negative** is the tone?
- Every post machine-translated into English, then scored with the Lexicoder Sentiment Dictionary: positive words minus negative words, per word
- Hypothesis: parties of the right talk more, and more negatively, about migration

> **Two dictionary decisions stacked: a translation model and a word list, and the paper's validity rests on both.**

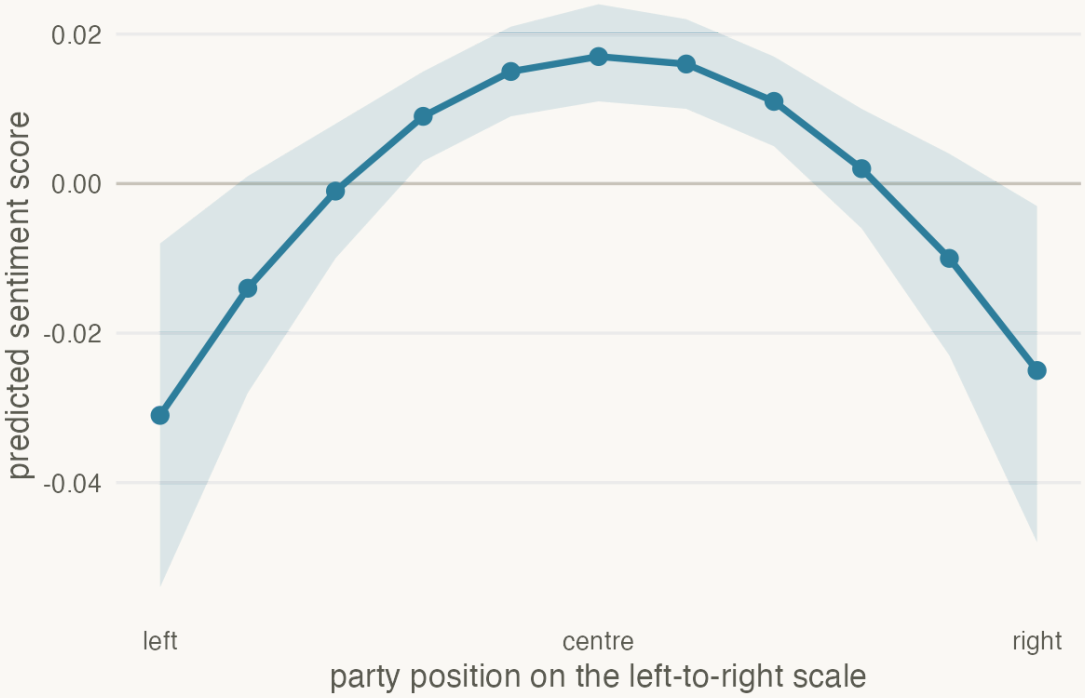
Heidenreich, T., Eberl, J.-M., Lind, F., & Boomgaarden, H. (2020). Political migration discourses on social media: A comparative perspective on visibility and sentiment across political Facebook accounts in Europe. *Journal of Ethnic and Migration Studies, 46*(7), 1261–1280. https://doi.org/10.1080/1369183X.2019.1665990

Young, L., & Soroka, S. (2012). Affective news: The automated coding of sentiment in political texts. *Political Communication, 29*(2), 205–231. https://doi.org/10.1080/10584609.2012.671234

Example 2 · Sentiment by ideological position

- Not what was hypothesised: the tone is most negative at **both** ends of the left–right scale
- Moderate parties in the middle are mildly positive; the extremes on either side are negative
- The visibility finding is the same shape: extremes post about migration more, whatever their side

> **The finding stands or falls with the list's ability to read a translated Facebook post. The paper says so in its limitations.**



Redrawn from Figure 2. Values read from the published figure, approximate. Heidenreich et al. (2020).

When a word list is the right tool

FOR

- Transparent: every rule can be read, and shared
- Cheap: no training data, runs on a laptop in seconds
- Deterministic: the same text gets the same label tomorrow
- Excellent for closed categories — names, actors, topics with a fixed vocabulary

AGAINST

- Low validity for anything that lives in context: sentiment, frames, irony
- Silent on texts that use other words; blind to negation
- A list built for one corpus and decade measures something else on another
- Encodes its makers' vocabulary, and their biases, as the construct

> **Each point on the right calls for a validation result, and no point on the left excuses you from producing one.**

Building your own

1. Define the category in words a coder could apply: the codebook first
2. Read keyword-in-context listings for candidate words; keep the ones that mean what you think they mean
3. Add spellings, inflections and lemmas; decide what to do with negation
4. Score a hand-coded sample; read the false positives and drop the words that cause them
5. Read the false negatives and find the words you missed
6. Repeat on a **fresh** sample — a list tuned on the texts it is scored against will not hold

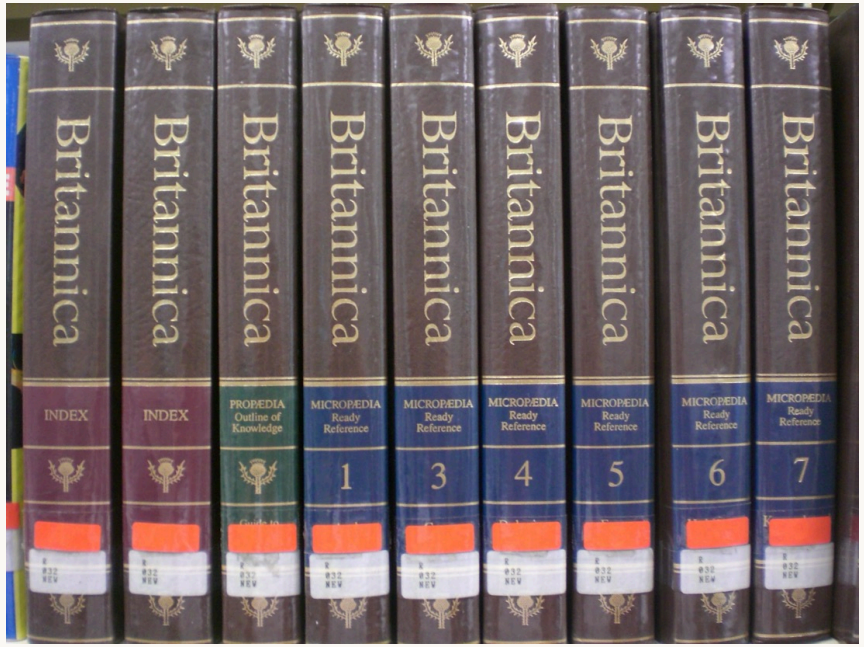
> **The Playground's build-a-dictionary is steps 4 to 6 on twenty texts, with precision and recall updating as you type.**

PLAYGROUND **Build-a-dictionary**

Conclusion: what a dictionary is

- A dictionary stores what people know about words: which words signal a category
- Applying it means looking words up, the way you look up an entry in a reference book
- So it is only as good as that knowledge: who wrote the list, for which texts, and when
- That is why it has to be validated on your own texts, against human judgement

> **A dictionary hands human knowledge about words to the computer. Validation checks whether that knowledge fits your texts.**



The *Encyclopædia Britannica*, one of the largest reference works in English: knowledge written down by people, to be looked up.

Photo: TSTPL Pedia (2007). *Britannica Micropedia Ready Reference Index 1–7*. Wikimedia Commons, CC BY-SA 3.0. https://commons.wikimedia.org/wiki/File:HK_Britannica_Micropedia_Ready_Reference_Index_1-7.JPG

V · This week

What you can do after this week.

- Turn a corpus into counts, weight them, and say what a count can and cannot claim
- Apply a dictionary as a join, and name the preprocessing decisions that change its result
- Build a gold standard: sample, codebook, independent coders, agreement
- Compute and read κ and α, and say why raw agreement is not reportable
- Read a confusion matrix and choose between accuracy, precision, recall and F1 for a given cost of error
- Say what class imbalance, silence and sample size do to those numbers

> **Everything in Part III applies unchanged to the classifier in week 3 and the language model in week 4. Only the instrument changes.**

This week

- **Tuesday · Lab A**: score the #corona tweets with a dictionary
- **Thursday · Lab B**: code 50 tweets with a partner, compute κ, then precision, recall and F1 on Yelp sentences
- **Sunday · homework 2**: Tuesday's Bing list against your gold standard, started in Lab B
- **Before Thursday · Playground**: *Two coders, one codebook* and *Build-a-dictionary*, ten minutes each
- **Reading**: van Atteveldt et al. (2021); *R for Computational Social Science*, text as data; Krippendorff (2019), ch. 11

> **The gold set you code on Thursday scores the dictionary in homework 2 and a language model in week 4. Code it with care.**

Questions?

> If anything is unclear, ask now, or on Tuesday, with a keyboard in front of you.

References

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement, 20*(1), 37–46. https://doi.org/10.1177/001316446002000104

Heidenreich, T., Eberl, J.-M., Lind, F., & Boomgaarden, H. (2020). Political migration discourses on social media: A comparative perspective on visibility and sentiment across political Facebook accounts in Europe. *Journal of Ethnic and Migration Studies, 46*(7), 1261–1280. https://doi.org/10.1080/1369183X.2019.1665990

Hu, M., & Liu, B. (2004). Mining and summarizing customer reviews. *Proceedings of the 10th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 168–177. https://doi.org/10.1145/1014052.1014073

Kahle, D. (2020). *sotu: United States presidential State of the Union addresses* (R package, version 1.0.2). https://CRAN.R-project.org/package=sotu

Kotzias, D., Denil, M., de Freitas, N., & Smyth, P. (2015). From group to individual labels using deep features. *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 597–606. https://doi.org/10.1145/2783258.2783380

Krippendorff, K. (2011). *Computing Krippendorff's alpha-reliability*. University of Pennsylvania. https://repository.upenn.edu/asc_papers/43

Krippendorff, K. (2019). *Content analysis: An introduction to its methodology* (4th ed.). SAGE.

Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics, 33*(1), 159–174. https://doi.org/10.2307/2529310

References (continued)

Mellado, C., Hallin, D., Cárcamo, L., Alfaro, R., Jackson, D., Humanes, M. L., … Ramos, A. (2021). Sourcing pandemic news: A cross-national computational analysis of mainstream media coverage of COVID-19 on Facebook, Twitter, and Instagram. *Digital Journalism, 9*(9), 1271–1295. https://doi.org/10.1080/21670811.2021.1942114

Mohammad, S. M., & Turney, P. D. (2013). Crowdsourcing a word–emotion association lexicon. *Computational Intelligence, 29*(3), 436–465. https://doi.org/10.1111/j.1467-8640.2012.00460.x

Silge, J., & Robinson, D. (2017). *Text mining with R: A tidy approach*. O'Reilly.

Stone, P. J., Dunphy, D. C., Smith, M. S., & Ogilvie, D. M. (1966). *The General Inquirer: A computer approach to content analysis*. MIT Press.

Van Atteveldt, W., Trilling, D., & Arcila, C. (2026). *R for computational social science* [Online book]. https://r4css.vanatteveldt.com/

Van Atteveldt, W., van der Velden, M. A. C. G., & Boukes, M. (2021). The validity of sentiment analysis: Comparing manual annotation, crowd-coding, dictionary approaches, and machine learning algorithms. *Communication Methods and Measures, 15*(2), 121–140. https://doi.org/10.1080/19312458.2020.1869198

Young, L., & Soroka, S. (2012). Affective news: The automated coding of sentiment in political texts. *Political Communication, 29*(2), 205–231. https://doi.org/10.1080/10584609.2012.671234